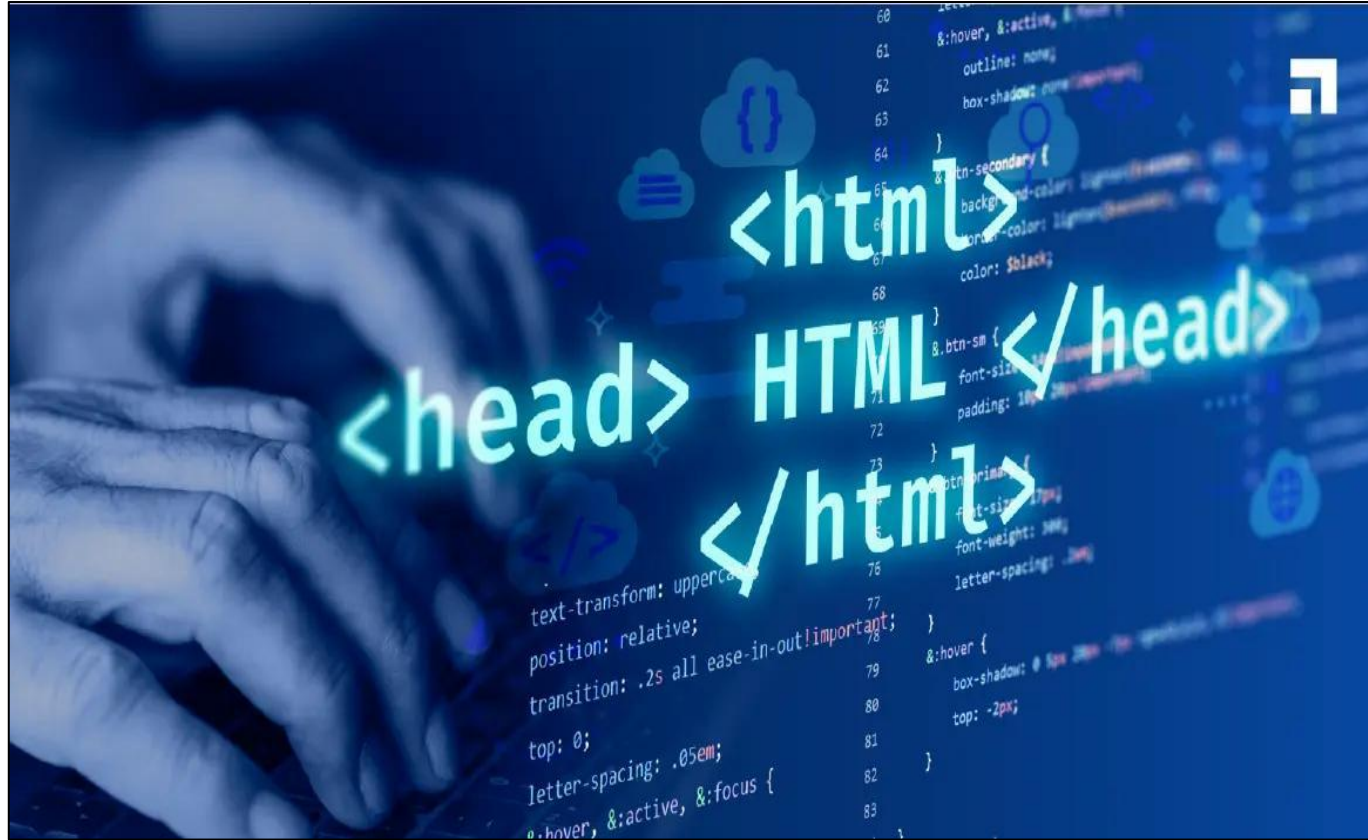
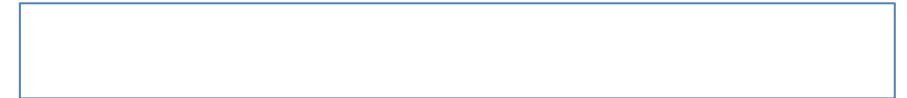




MODULE
4

JavaScript

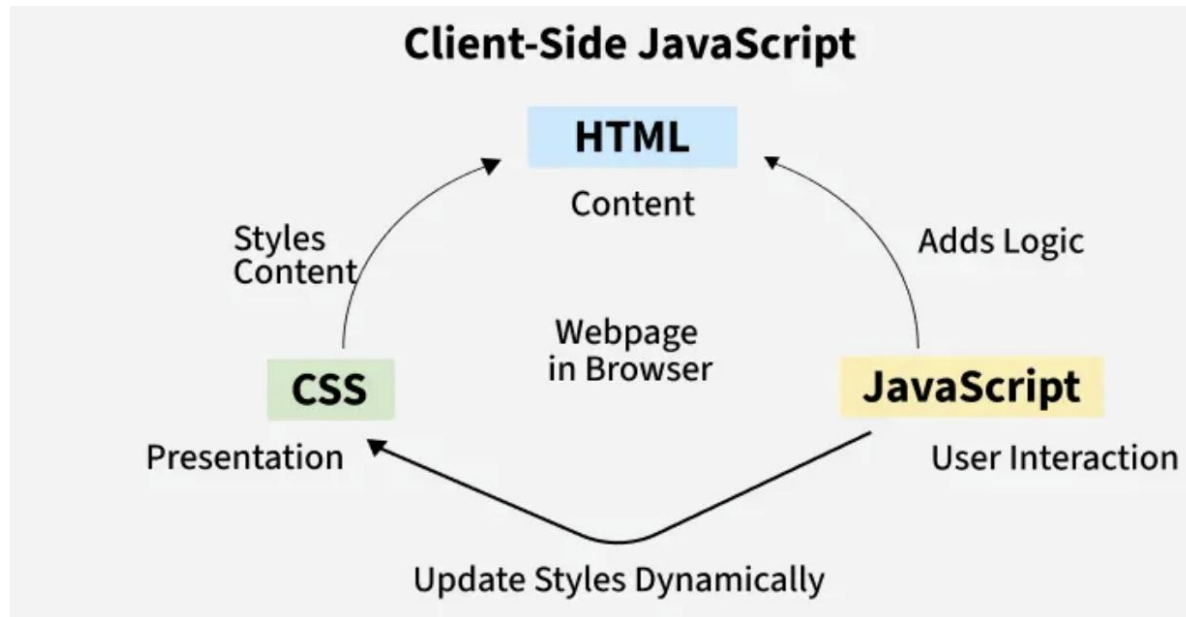




Java Script

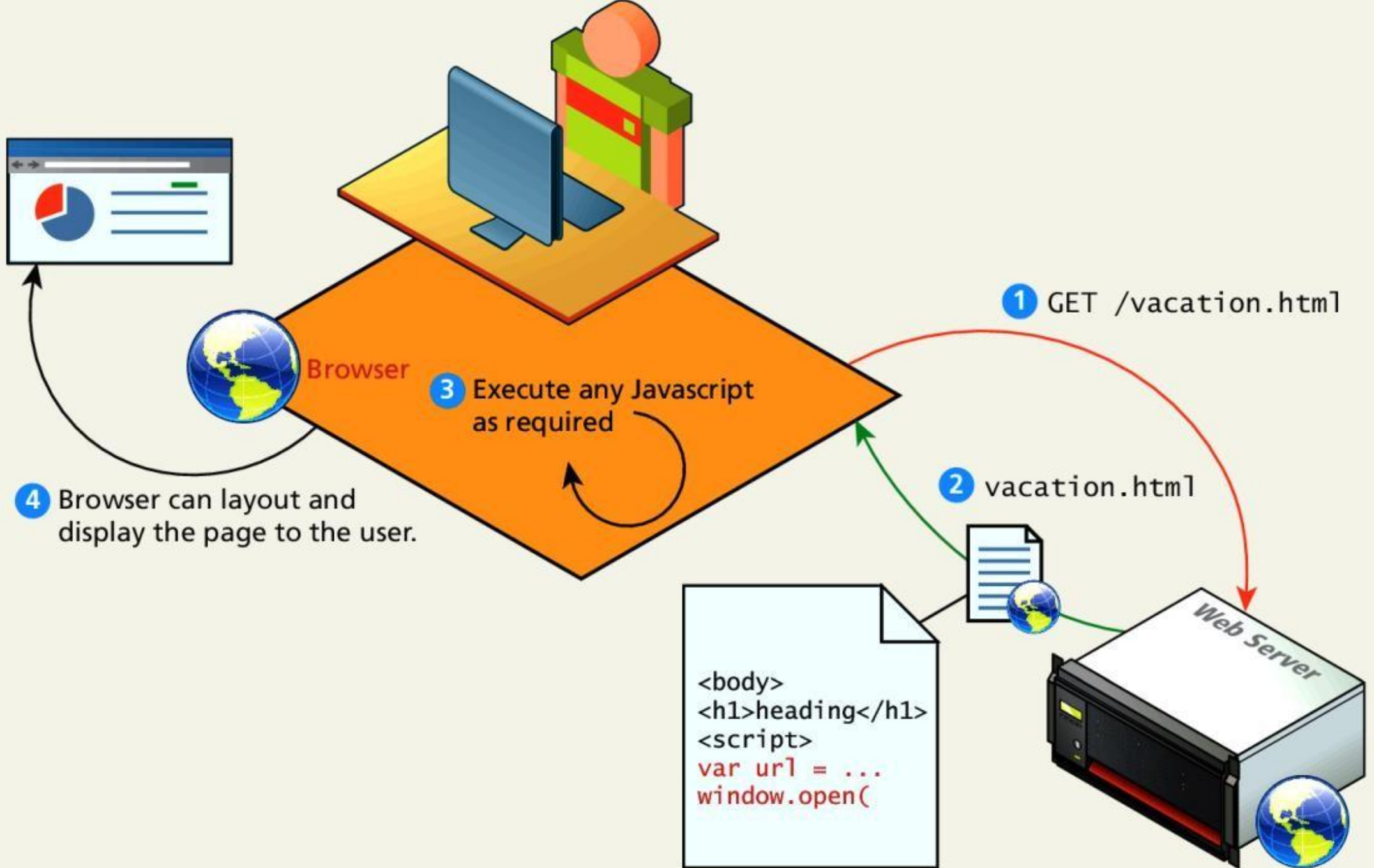
- **JavaScript** is a programming language commonly used in web development.
- It was originally **developed by Netscape** as a means to add dynamic and interactive elements to websites.
- JavaScript is a **client-side scripting language**, which means the source code is processed by the client's web browser rather than on the web server.
- JavaScript can **calculate, manipulate and validate data**.
- **JavaScript in body or head**: Scripts can be placed inside the body or the head section of an HTML page or inside both head and body.

- JavaScript is a programming language used to create dynamic content for websites.
- **Client Side:** On the client side, JavaScript works along with HTML and CSS. HTML adds structure to a web page, CSS styles it, and JavaScript brings it to life by allowing users to interact with elements on the page, such as actions on clicking buttons, filling out forms, and showing animations. JavaScript on the client side is directly executed in the user's browser.
- **Server Side:** On the Server side (on Web Servers), JavaScript is used to access databases, file handling, and security features to send responses, to browsers.



JAVASCRIPT

- JavaScript runs right inside the browser
- JavaScript is dynamically typed
- JavaScript is object oriented in that almost everything in the language is an object
- web standard (but not supported identically by all browsers)
- NOT related to Java other than by name and some syntactic similarities



CLIENT-SIDE SCRIPTING

- ❧ Processing can be offloaded from the server to client machines, thereby reducing the load on the server.
- ❧ The browser can respond more rapidly to user events than a request to a remote server ever could, which improves the user experience.
- ❧ JavaScript can interact with the downloaded HTML in a way that the server cannot, creating a user experience more like desktop software than simple HTML ever could

LINKING TO A JAVASCRIPT FILE: SCRIPT

- ❧ Script tag should be placed in HTML page's head ``
- ❧ Script code is stored in a separate .Js file ``
- ❧ JS code can be placed directly in the HTML file's body or head (like CSS)

LINKING TO A JAVASCRIPT FILE: SCRIPT

Using **<Script>** tag

<html>

<head>

<script src="filename.js" type="text/javascript">

</script>

</head>

</html>

WHERE DOES JAVASCRIPT GO?

JavaScript can be linked to an HTML page in a number of ways.

- 🔗 Inline
- 🔗 Embedded
- 🔗 External

INLINE JAVASCRIPT

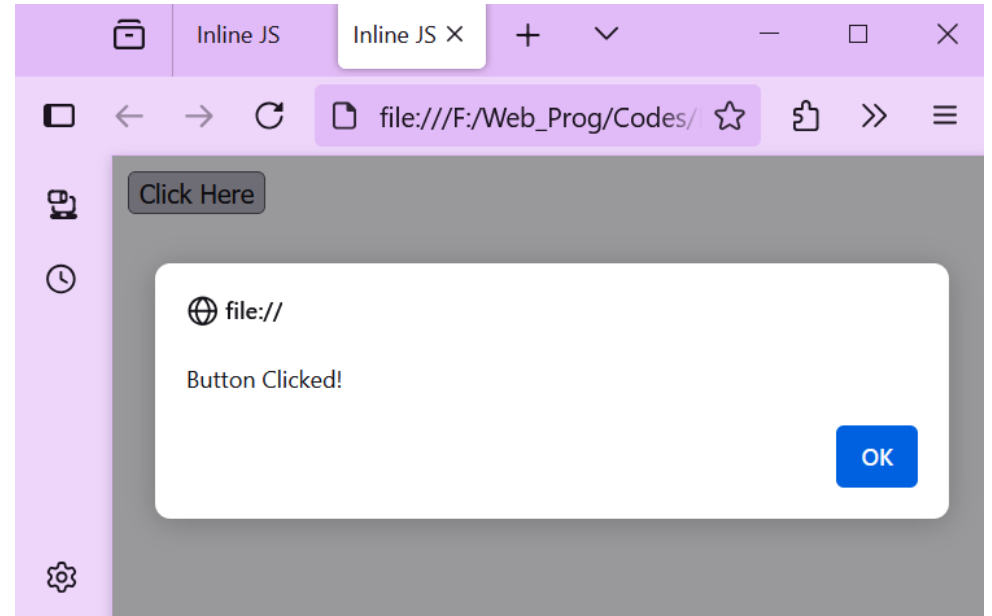
Inline JavaScript refers to the practice of including JavaScript code directly within certain HTML attributes.

```
<a href="JavaScript:OpenWindow();"more info</a>  
<input type="button" onclick="alert('Are you sure?');" />
```

LISTING 6.1 Inline JavaScript example

SAMPLE CODE

```
<html>
<head>
<title> Inline JS</title>
</head>
<body>
<button onclick="alert(' Button Clicked!')">
    Click Here
</button>
</body>
</html>
```



EMBEDDED JAVASCRIPT

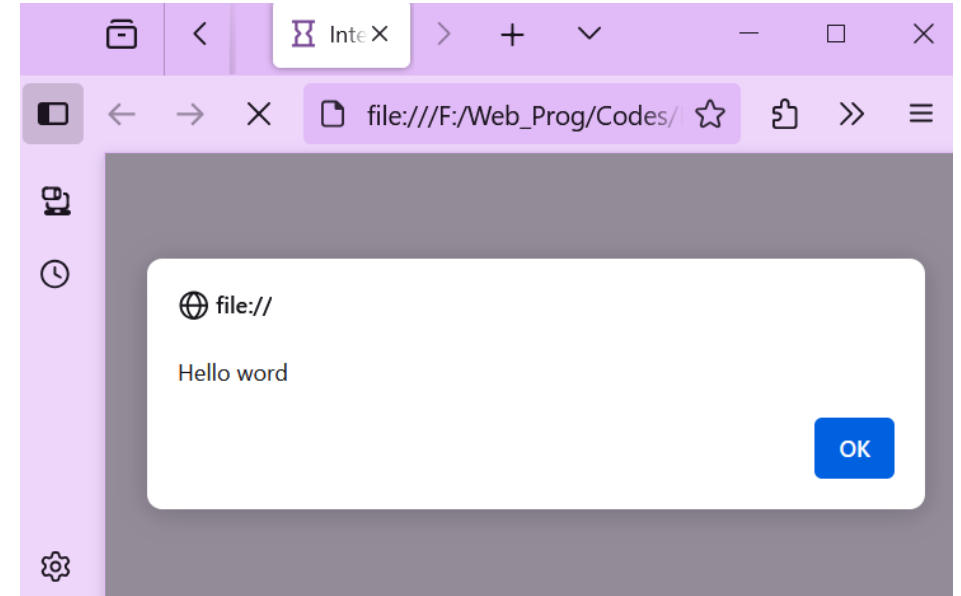
Embedded JavaScript refers to the practice of placing JavaScript code within a `<script>` element

```
<script type="text/javascript">
/* A JavaScript Comment */
alert ("Hello World!");
</script>
```

LISTING 6.2 Embedded JavaScript example

SAMPLE CODE

```
<html>
<head>
<title> Internal JS</title>
<script type="text/javascript">
    Alert("Hello World");
</script>
</head>
<body>
</body>
</html>
```



EXTERNAL JAVASCRIPT

- ✧ JavaScript supports this separation by allowing links to an external file that contains the JavaScript.
- ✧ By convention, JavaScript external files have the extension **.is**

```
<head>  
  <script type="text/JavaScript" src="greeting.js">  
  </script>  
</head>
```

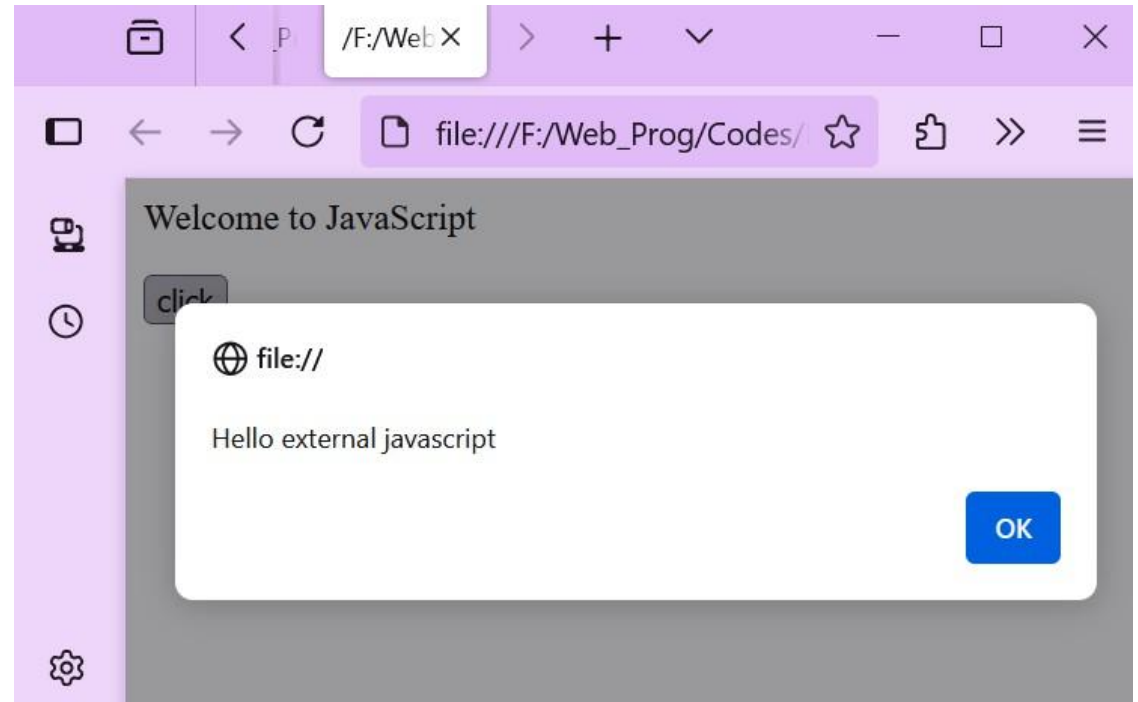
LISTING 6.3 External JavaScript example

SAMPLE CODE

```
<html>
<head>
<script type="text/javascript"
src="ext.js"></script>
</head>
<body>
<p>Welcome to JavaScript</p>
<form>
<input type="button" value="click"
onclick="msg()" />
</form>
</body>
</html>
```

ext.js

```
function msg(){
  alert("Hello external javascript");
}
```



JAVASCRIPT - BASICS

- Variables,
- Assignment Statements
- Pop up boxes

Hello World Web Page

- Three ways to print 'hello world' program in javascript

- **console.log()**
- **alert()**
- **document.write()**

1. Using console.log()

- Prints the message in the **browser console**
- Mainly used for testing and debugging
- Not visible** on the web page



```
<html> <head></head>
<body>
<h1>Check the console for the message!</h1>
  <script>
    console.log("Hello, World!");
  </script>
</body> </html>
```

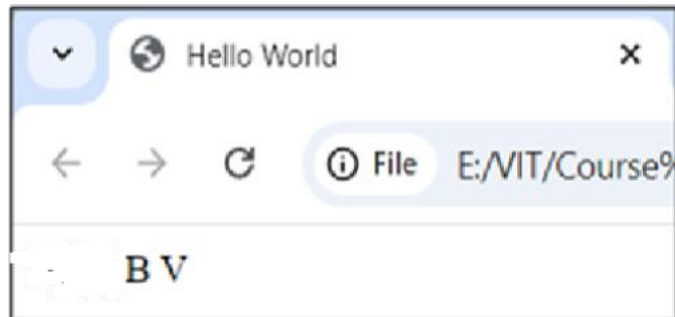
Check the console
for the message!



2. Using document.write()

- Used to display **text directly on the web page**

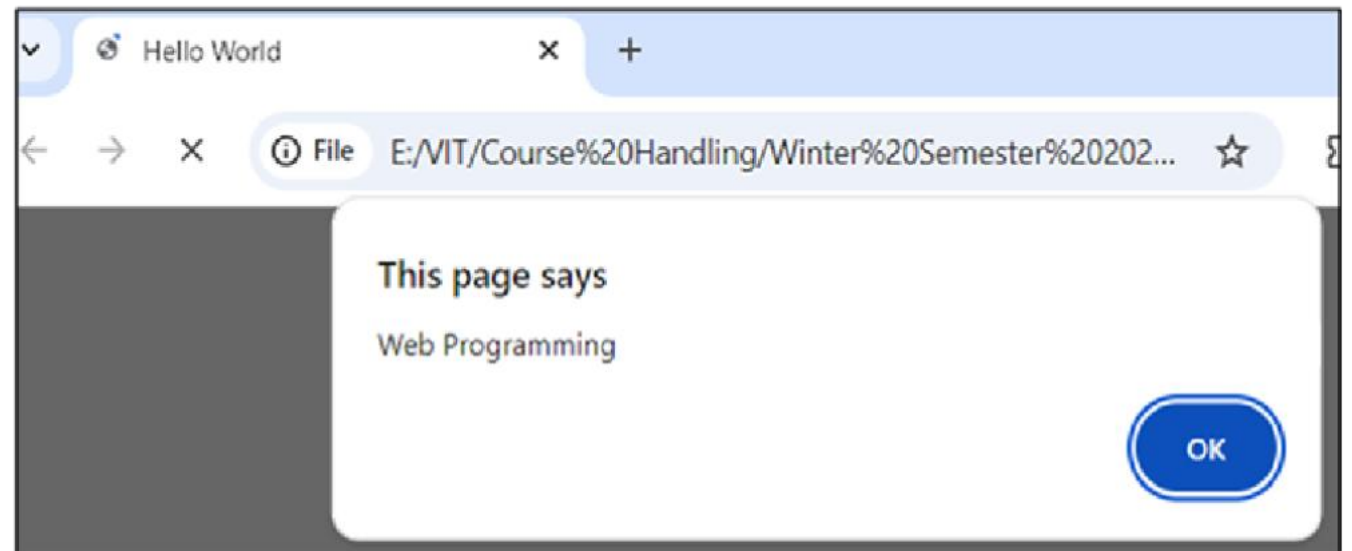
```
<!DOCTYPE html>
<html>
<head>
  <title>Hello World</title>
</head>
<body>
<script>
  document.write("B V");
</script>
</body>
</html>
```



3. Using alert()

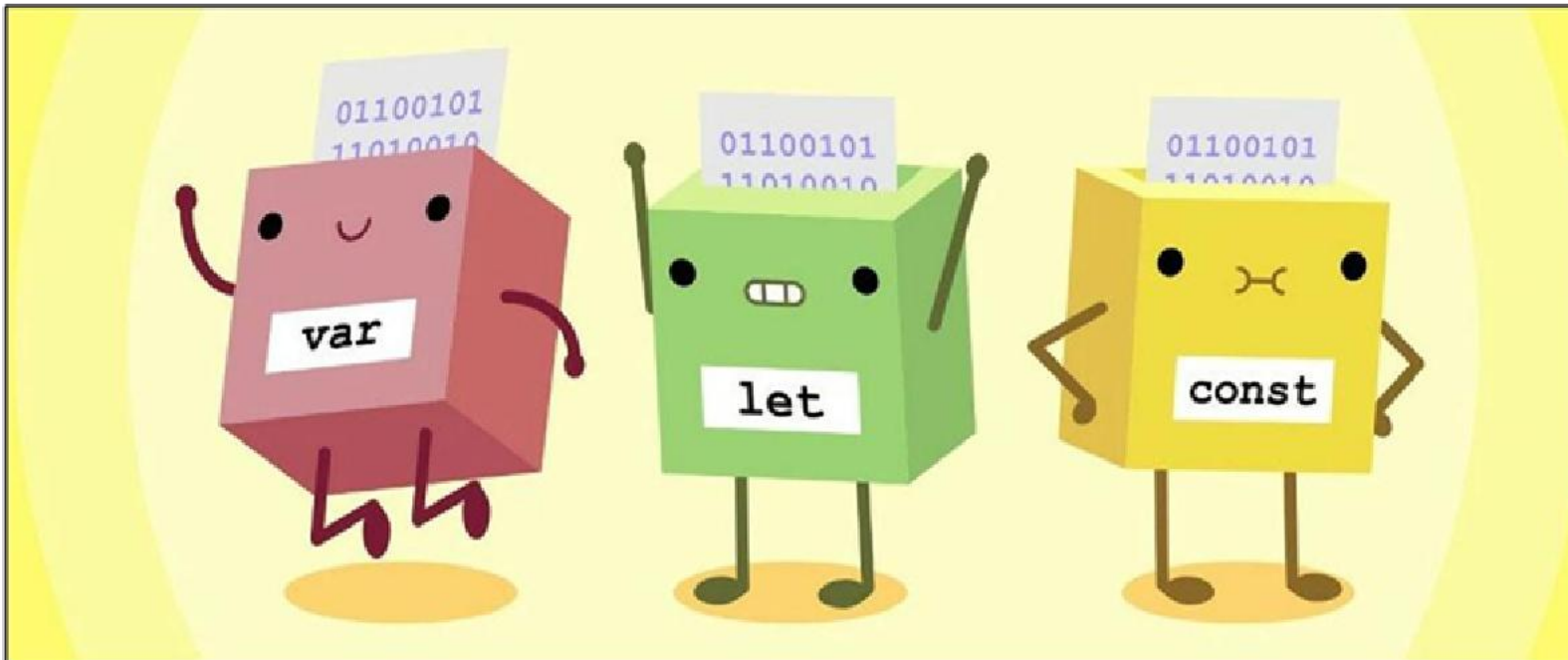
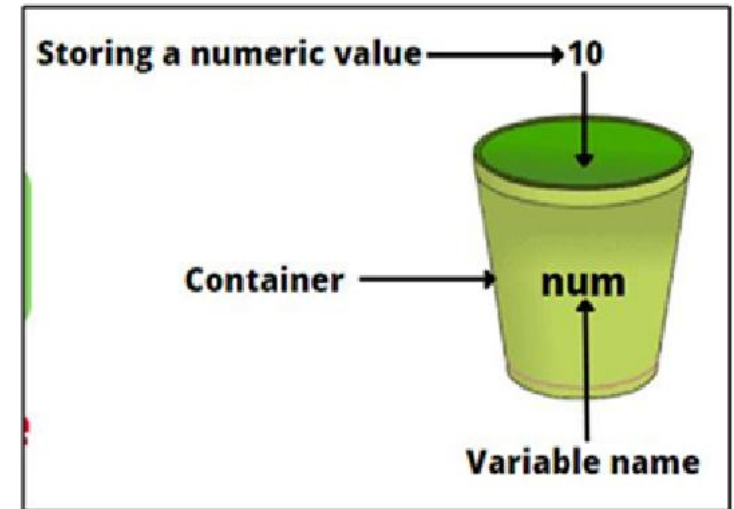
- Used to show a **pop-up message box** on the screen.

```
<html> <head>
  <title>Hello World</title>
</head>
<body>
<script>
  alert("Web Programming");
</script>
</body> </html>
```



Variables

- A variable is like a container that holds data that can be reused or updated later in the program.



1. var Keyword

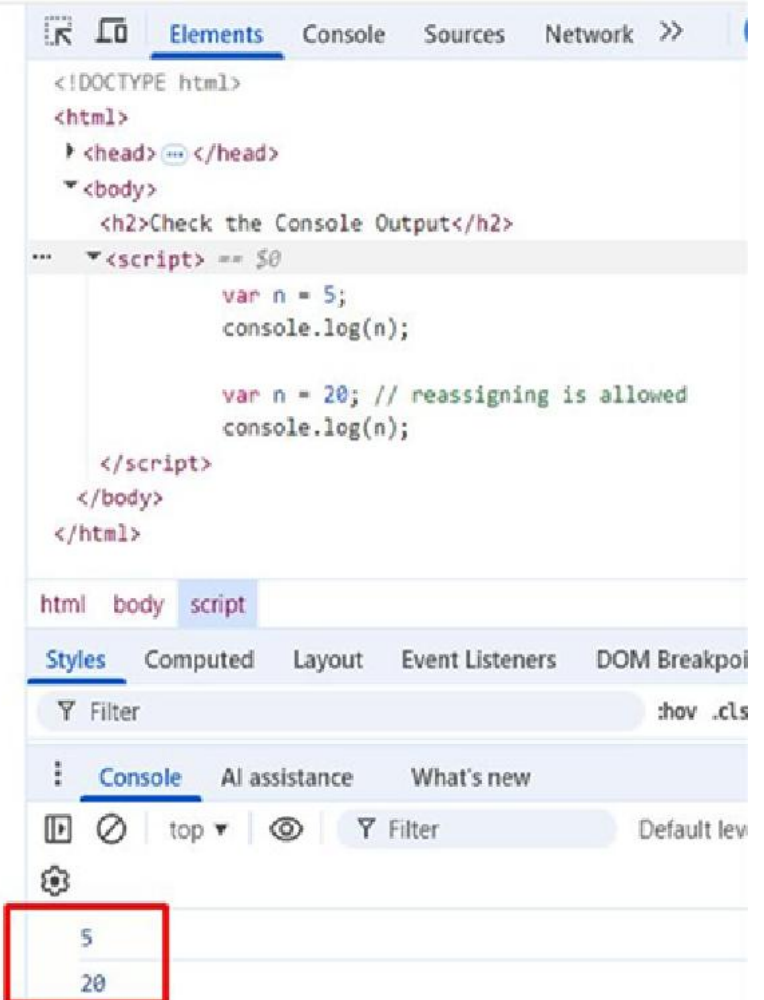
- The var keyword is used to declare a variable.
- Has a **function-scoped** or globally-scoped behaviour.
- Can be **redeclared**



```
var age = 25;  
var age = 30; // allowed  
age = 35;    // allowed
```

```
<html> <head>  
</head>  
<body>  
  <h2>Check Console Output</h2>  
  <script>  
    var n = 5;  
    console.log(n);  
  
    var n = 20;  
    console.log(n);  
  </script>  
</body> </html>
```

Check the Console Output



2. let Keyword

- let is **block-scoped**, which means it works only inside the **block { }**, **if** or **for** where it is declared.
- It **cannot be redeclared** in the same scope

```
let name = "John";  
name = "Doe"; // allowed  
let name = "Smith"; // ❌ not allowed
```

```
<html> <body>  
<h2>Let Example</h2>  
<p id="result"></p>  
<script>  
  let x = 10;  
  x = 20;  
  
  document.getElementById("result").innerText = x;  
</script>  
</body> </html>
```

getElementById("result")

- Searches the page for an element with id="result"

innerText

- Refers to the visible text content inside that element.

```
if (true)  
{  
  let a = 5;  
}  
console.log(a);
```

Let Example

20

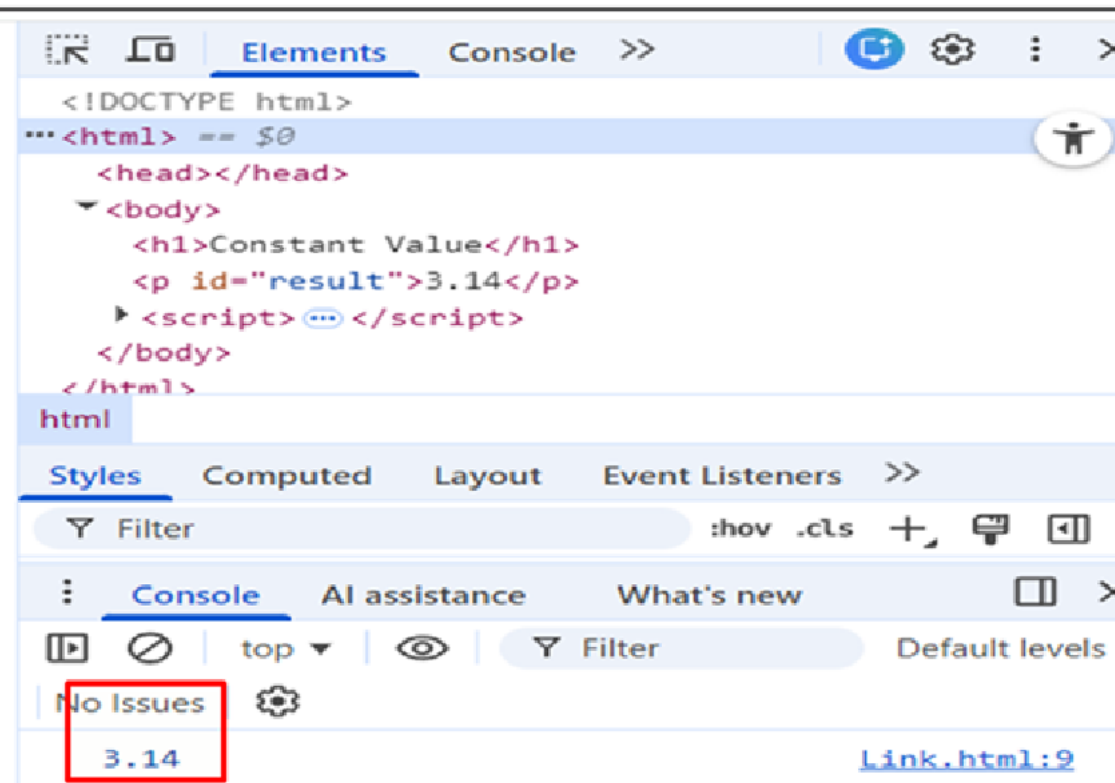
3. const Keyword

- const is a keyword used to *declare a constant variable*.
- Value *cannot be changed* once it is assigned.

```
<html> <body>
<h1>Constant Value</h1>
<p id="result"></p>
<script>
  const PI = 3.14;
  console.log(PI);
  document.getElementById("result").innerText = PI;
</script>
</body> </html>
```

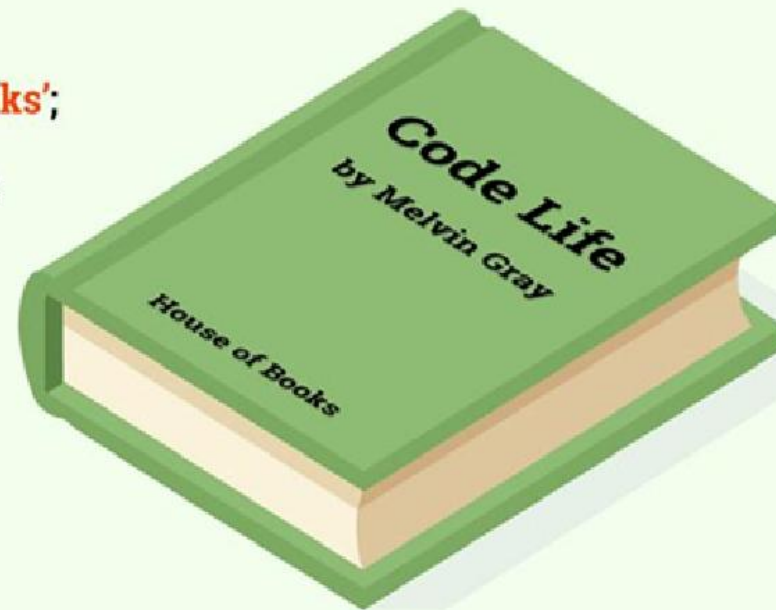
**Constant
Value**

3.14



Feature	var	let	const
Scope	Function	Block	Block
Can reassign?	Yes	Yes	No
Can redeclare?	Yes	No	No
Must initialize?	No	No	Yes
Use today?	Avoid	Yes	Best

```
var title = 'Code Life';  
let publisher = 'House of Books';  
const author = 'Melvin Gray';
```



var

var apple = 



a thing in a box
named "apple"

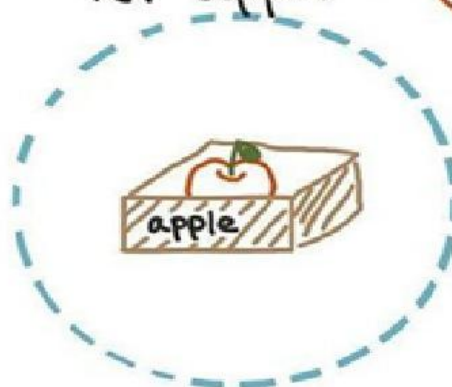
apple = 



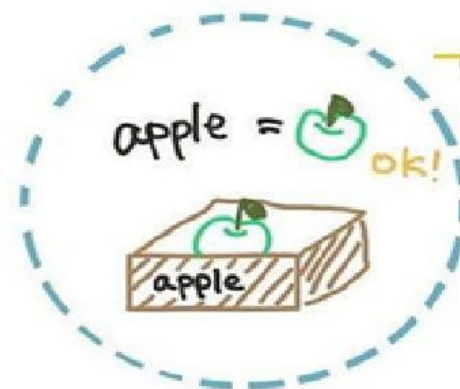
you can swap
item later

let

let apple = 



a thing in a box
named "apple" w/
protection shield



~~apple =  NG~~

you can swap item
only if you ask
inside of the shield

const

const apple = 



a thing in
LOCKED cage
named "apple"



~~apple =  NG~~

you can't
swap item
later.



apple.multiply(3) OK!

... but you can ask
the item to change itself
(if the item has method
to do that)

A company payroll portal generates an employee salary slip. The portal stores employee details like name, department, month, salary components, and calculates totals. The system should show a detailed salary summary to the employee using Console, Alert, and Document output methods.

Write a JavaScript program that:

1) Create variables:

Employee details: **(Console Output)**

- employee = "Sneha" (const)
- empId = "EMP1023" (const)
- department = "HR" (const)
- month = "January" (const)

Salary components: **(let)**

- basic = 25000 , bonus = 5000
- hra = 8000, travelAllowance = 2000

Deductions: **(let)**

- pf = 1800
- tax = 1200

2) Calculate:

grossSalary = basic + bonus + hra + travelAllowance

totalDeductions = pf + tax

netSalary = grossSalary - totalDeductions **(alert)**

Document Output : Salary slip generated successfully for employee

JavaScript Variables

What are Variables?

- Variables are containers for storing data (storing data values).
- In a programming language, variables are used to store data values.
- JavaScript uses the **keywords var, let and const to declare variables.**
- An equal sign is used to assign values to variables.

```
let x;
```

```
x = 6;
```

JavaScript Variables

Ways to Declare a JavaScript Variable:

- Using var
- Using let
- Using const

- Always declare JavaScript variables with var, let, or const.
- The var keyword is used in all JavaScript code from 1995 to 2015.
- The let and const keywords were added to JavaScript in 2015.
- code to run in older browser, must use var.
- a general rule: always declare variables with const.
- **If the value of the variable can change, use "let"**

Re-Declaring JavaScript Variables

- **re-declare a JavaScript variable declared with var, it will not lose its value.**

```
var carName = "Volvo";  
var carName;
```

- **cannot re-declare a variable declared with let or const.**

- will not work:

```
let carName = "Volvo";  
let carName;
```

- If you put a number in quotes, the rest of the numbers will be treated as strings and concatenated.

```
let x = 2 + 3 + "5";
```

Result : 55

JavaScript Let

- The let keyword was introduced in ES6 (2015).
- **Variables defined with let cannot be Redeclared.**
- **Variables defined with let must be Declared before use.**
- Variables defined with let **have Block Scope.**

Cannot be Redeclared

```
let x = "John Doe";
```

```
let x = 0;
```

```
// SyntaxError: 'x' has already  
    been declared
```

With var you can redeclare

```
var x = "John Doe";
```

```
var x = 0;
```

Block Scope

- Before 2015, JavaScript had only Global Scope and Function Scope.
- Two important new JavaScript keywords: **let and const were introduced.**
- These two keywords **provide Block Scope** in JavaScript.
- **Variables declared inside a { } block cannot be accessed from outside the block**

Example

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

Variables declared with the var keyword can NOT have block scope.

```
{  
  var x = 2;  
}  
// x CAN be used here
```

Redeclaring Variables

- Redeclaring a variable using the var keyword can impose problems.
- **Redeclaring a variable inside a block will also redeclare the variable outside the block**

```
var x = 10;  
// Here x is 10  
{  
var x = 2;  
// Here x is 2  
}  
// Here x is 2
```

- Redeclaring a variable using the let keyword can solve this problem.
- Redeclaring a variable inside a block will not redeclare the variable outside the block

```
let x = 10;  
// Here x is 10  
{  
  let x = 2;  
  // Here x is 2  
}  
// Here x is 10
```

- Redeclaring a variable with let, in another block, IS allowed

```
let x = 2;  // Allowed
```

```
{  
let x = 3;  // Allowed  
}
```

```
{  
let x = 4;  // Allowed  
}
```

JavaScript Const

- The const keyword was introduced in ES6 (2015).
- Variables defined with const cannot be Redeclared.
- Variables defined with const cannot be Reassigned.
- Variables defined with const have Block Scope.

Cannot be Reassigned

- A const variable cannot be reassigned

```
const PI = 3.141592653589793;
```

```
PI = 3.14;    // This will give an error
```

```
PI = PI + 10; // This will also give an error
```

Must be Assigned

- JavaScript const variables must be assigned a value when they are declared

```
//correct
```

```
const PI = 3.14159265359;
```

```
//Incorrect
```

```
const PI;
```

```
PI = 3.14159265359;
```

Block Scope

- Declaring a variable with const is similar to let when it comes to Block Scope.
- The x declared in the block, in this example, is not the same as the x declared outside the block

```
const x = 10;
```

```
// Here x is 10
```

```
{
```

```
const x = 2;
```

```
// Here x is 2
```

```
}
```

```
// Here x is 10
```

Assignment Statements

- Assignment operators assign values to JavaScript variables.

Basic Assignment

```
let x = 10;
```

Reassignment

- Variables declared with `let` or `var` can be changed later.

```
let score = 50;  
score = 75;
```

Constant Assignment

- `const` variables cannot be reassigned.

```
const pi = 3.14;
```

Assignment with Expressions

- The right side can be a calculation.

```
let a = 5;  
let b = 3;  
let sum = a + b;
```

Chained Assignment

```
let a, b, c;  
a = b = c = 100;
```

Assignment by Value vs Reference

```
let a = 10;  
let b = a;  
b = 20;
```

Variables and Datatype

- Primitive types used in javascript

Number(x=10)

String(s="hello")

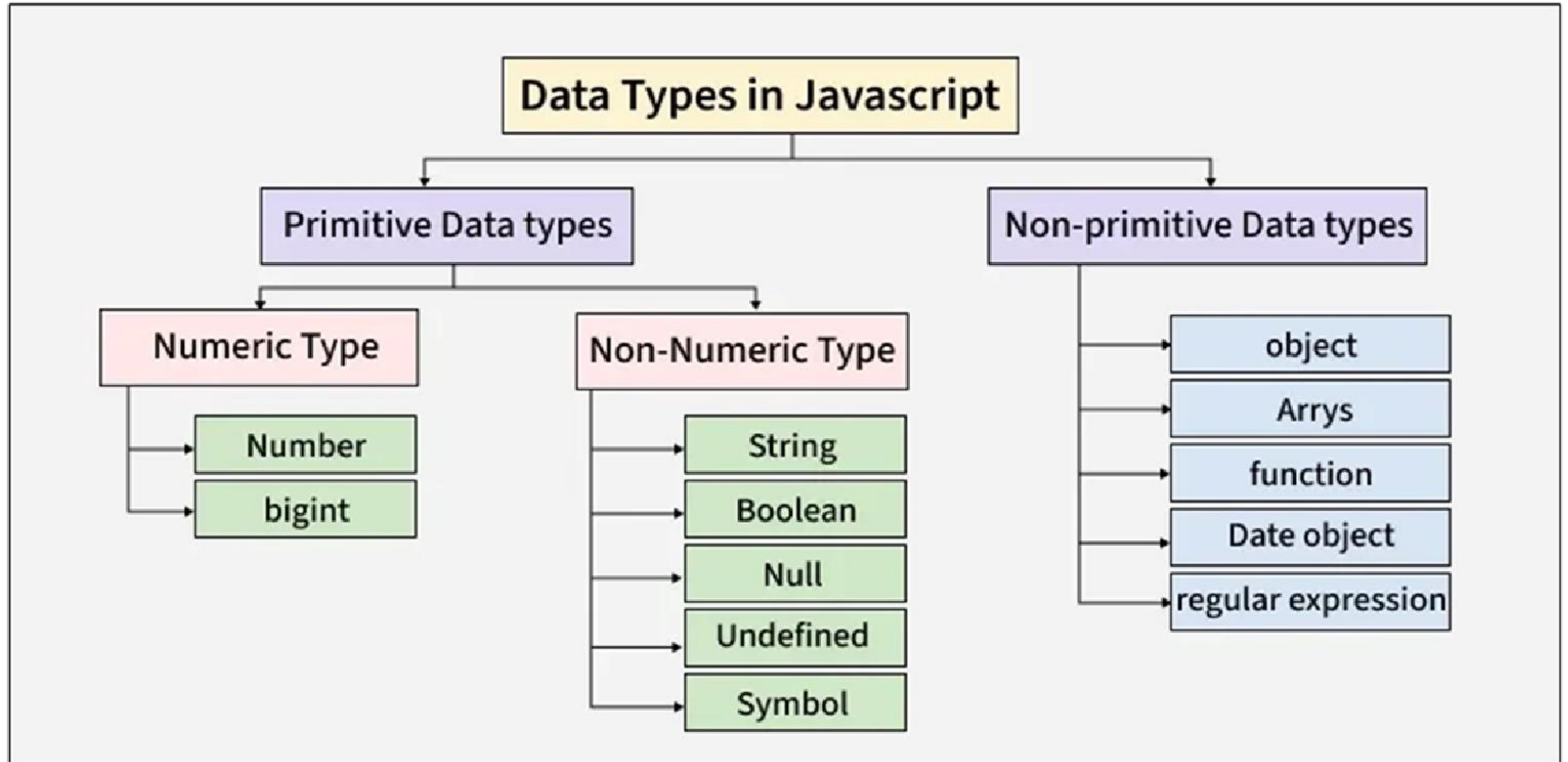
Boolean

Undefined(explicitly declared but not assigning value)

Null(no value)

- Unlike its predecessor languages C, C++ and Java, JavaScript does not require variables to have a declared type before they can be used in a script.
- A variable in JavaScript can contain a value of any data type, and in many situations JavaScript automatically converts between values of different types for you.
- For this reason, JavaScript is referred to as a loosely typed language.

Data Types



Primitive Data Types

Category	Data Type	Description	Example
Numeric	Number	Represents integers & floating-point numbers	let x = 10;
	BigInt	Represents very large integers beyond safe limit	let y = 12345678901234567890n;
Non-Numeric	String	Represents text or sequence of characters	let name = "John";
	Boolean	Represents logical values: true or false	let isValid = true;
	Null	Represents intentional absence of value	let data = null;
	Undefined	Variable declared but not assigned	let a;
	Symbol	Represents a unique identifier	let id = Symbol("id");

Non-Primitive (Reference) Data Types

Data Type	Description	Example
Object	Collection of key-value pairs	<code>let user = { name: "Alice", age: 25 };</code>
Array	Ordered list of values	<code>let arr = [1, 2, 3];</code>
Function	Block of reusable code	<code>function greet() { return "Hi"; }</code>
Date Object	Represents date and time	<code>let d = new Date();</code>
Regular Expression	Used for pattern matching in strings	<code>let pattern = /abc/;</code>

KEYWORDS

Keywords are part of JavaScript's syntax and control how the program is written and executed.

abstract	double	in	super*
arguments	else	instanceof	switch
await*	enum*	int	synchronized
boolean	eval	interface	this
break	export*	let*	throw
byte	extends*	long	throws
case	false	native	transient
catch	final	new	true
char	finally	null	try
class*	float	package	typeof
const	for	private	var
continue	function	protected	void
debugger	goto	public	volatile
default	if	return	while
delete	implements	short	with
do	import*	static	yield

JavaScript Comments

- **Not all JavaScript statements are "executed".**
- Code after **double slashes //** or **between /* and */** is treated as a comment.
- Comments **are ignored and** will not be executed.

JavaScript Syntax

- JavaScript syntax is the set of rules, how JavaScript programs are constructed

JavaScript Values:

- The JavaScript syntax **defines two types of values:**
 - **Fixed values**
 - **Variable values**
- Fixed values are called **Literals.**
- Variable values are called **Variables.**

JavaScript Literals

The two most important syntax rules for fixed values are:

1. **Numbers** are written with or without decimals

10.50

1001

2. **Strings** are text, written within double or single quotes

"John Doe"

'John Doe'

JavaScript Identifiers / Names

- **Identifiers are JavaScript names.**
- **Identifiers are used to name variables and keywords, and functions.**
- The rules for legal names are the same in most programming languages.
- A JavaScript name must begin with:
 - **A letter (A-Z or a-z)**
 - **A dollar sign (\$)**
 - **Or an underscore (_)**
- Subsequent characters may be letters, digits, underscores, or dollar signs.

JavaScript is Case Sensitive

- **All JavaScript identifiers are case sensitive.**
- The variables **lastName** and **lastname**, are two different variables.

JavaScript and Camel Case

Programmers have used different ways of joining multiple words into one variable name:

- **Hyphens:**
 - first-name, last-name, master-card, inter-city.
 - **Hyphens are not allowed in JavaScript. They are reserved for subtractions.**
- **Underscore:** first_name, last_name, master_card, inter_city.
- **Upper Camel Case (Pascal Case):** FirstName, LastName, MasterCard, InterCity.
- **Lower Camel Case:** JavaScript programmers tend to use camel case that starts with a lowercase letter:
 - firstName, lastName, masterCard, interCity.

JavaScript Character Set

- JavaScript uses the **Unicode** character set.
- Unicode covers (almost) all the characters, punctuations, and symbols in the world.

Simple Javascript program

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset = "utf-8">
```

```
<title>A First Program in JavaScript</title>
```

```
<script type = "text/javascript">
```

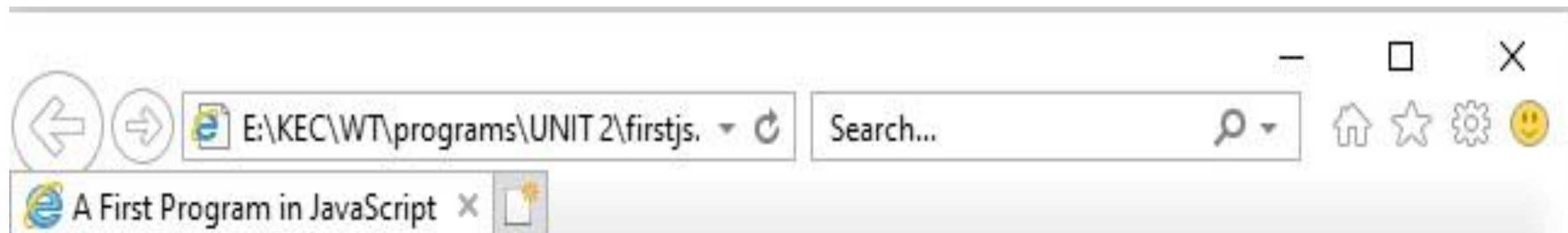
```
document.writeln("<h1>Welcome to JavaScript Programming!</h1>");
```

```
</script>
```

```
</head>
```

```
<body></body>
```

```
</html>
```



Welcome to JavaScript Programming!

Example 2(javascript and CSS)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset = "utf-8">
```

```
<title>A First Program in JavaScript</title>
```

```
<script type = "text/javascript">
```

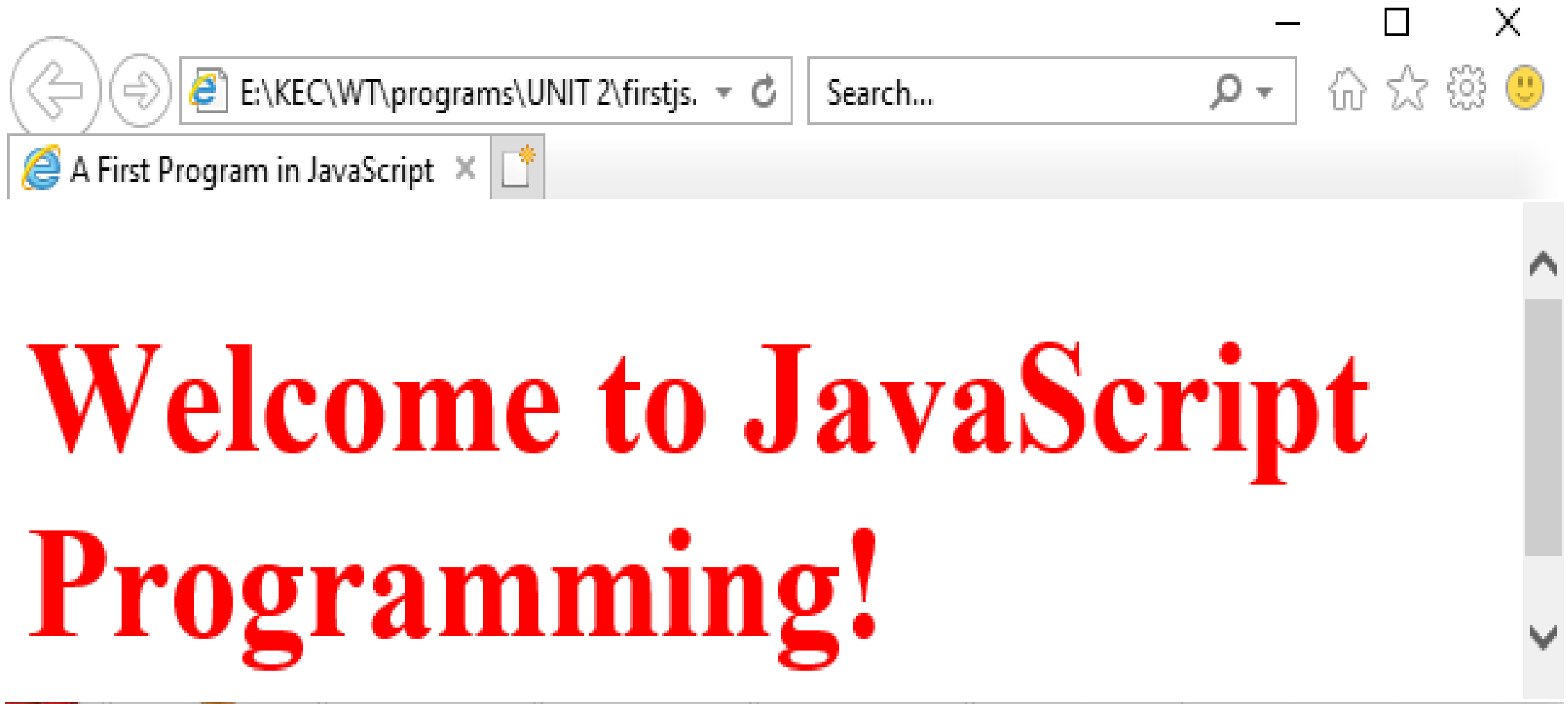
```
document.writeln("<h1 style='font-size:56px;color:red'>Welcome to JavaScript  
Programming!</h1>");
```

```
</script>
```

```
</head>
```

```
<body></body>
```

```
</html>
```

Operators

- Arithmetic Operators(+,-,*,/,%)
- Relational Operators(<,>,<=,>==,!=)
- Logical Operators(&&||)
- Assignment Operators(=)
- Increment Operators
- Decrement Operators

String Concatenation Operator(+)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset = "utf-8">
```

```
<title>string concatenation</title>
```

```
<script type = "text/javascript">
```

```
var str="programming"
```

```
document.write("<h1>Web"+str+"concepts</h1>");
```

```
</script>
```

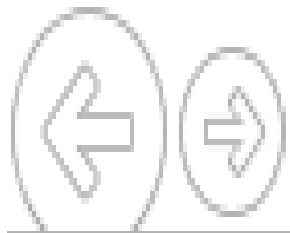
```
</head><body></body></html>
```



Webprogrammingconcepts

Addition of two numbers

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>addition</title>
<script type = "text/javascript">
var  a=10;
var  b=20;
var c=a+b;
document.write("the sum of a and b is"+c);
</script>
</head>
<body></body>
</html>
```



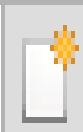
E:\KEC\WT\programs\UNI ▾ ↻



Search...



addition



the sum of a and b is30

popup boxes

- JavaScript has three kind of popup boxes: **Alert box, Confirm box, and Prompt box.**
- **When an alert box pops up**, the user will have to click "OK" to proceed.
- **When a confirm box pops up**, the user will have to click either "OK" or "Cancel" to proceed. If the user clicks "OK", the box returns true. If the user clicks "Cancel", the box returns false
- **When a prompt box pops up**, the user will have to click either "OK" or "Cancel" to proceed after entering an input value. If the user clicks "OK" the box returns the input value. If the user clicks "Cancel" the box returns null.

Displaying Text in an Alert Dialog

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset = "utf-8">
```

```
<title>alert box</title>
```

```
<script type = "text/javascript">
```

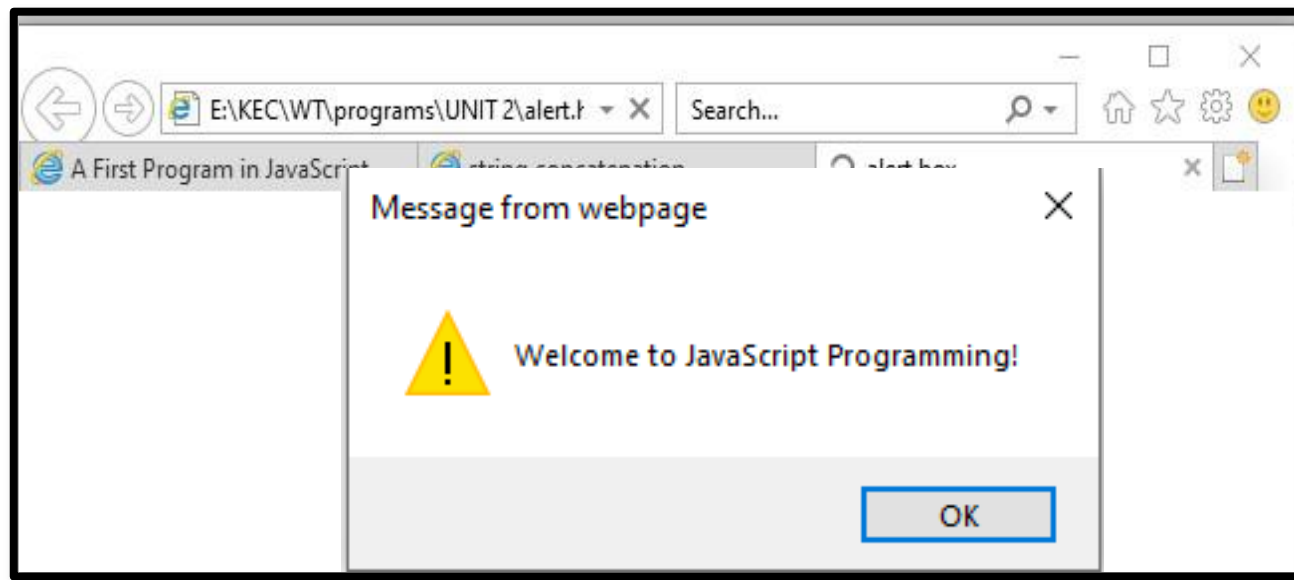
```
window.alert( "Welcome to JavaScript Programming!");
```

```
window.confirm("Do you want to continue");</script>
```

```
</head>
```

```
<body></body>
```

```
</html>
```

Obtaining User Input with prompt Dialogs

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>dynamic page</title>
<script type = "text/javascript">
var name;
name=window.prompt("enter your name");
document.write("<h1>welcome "+name+"</h1>");
</script>
</head>
<body></body>
</html>
```



Adding Integers using Prompt box

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>adding integers</title>
<script type = "text/javascript">
var firstnumber;
var secondnumber;
firstnumber=window.prompt("enter your first number");
secondnumber=window.prompt("enter your second number");
a=parseInt(firstnumber);
b=parseInt(secondnumber);
var c=a+b;
document.write("the result is"+c);
</script>
</head><body></body>
</html>
```

Explorer User Prompt

Script Prompt:

enter your first number

10

OK

Cancel

Explorer User Prompt

Script Prompt:

enter your second number

20

OK

Cancel



E:\KEC\WT\programs\UNI



Search...



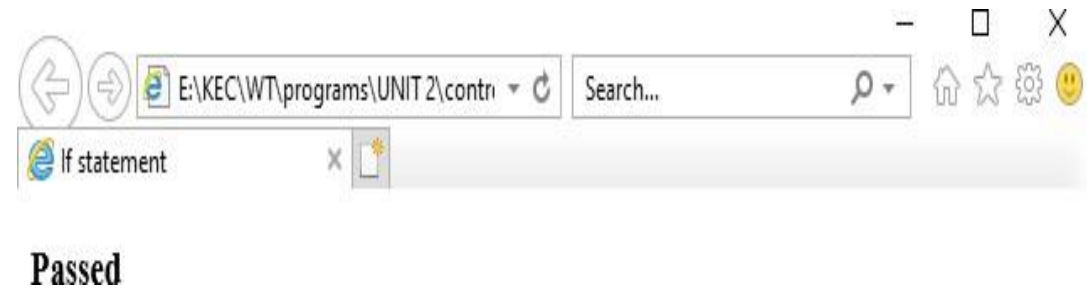
adding integers



the result is30

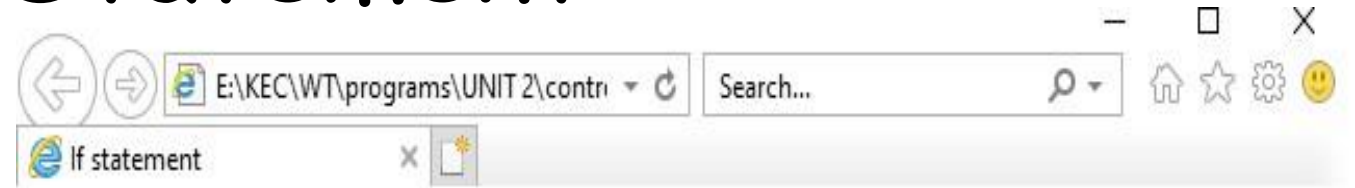
Control Statements I : if Selection Statement

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>If statement</title>
<script type = "text/javascript">
grade=70;
if ( grade >= 60 )
document.writeln( "<h3>Passed</h3>" );
</script>
<body>
</body>
</html>
```



if...else Selection Statement

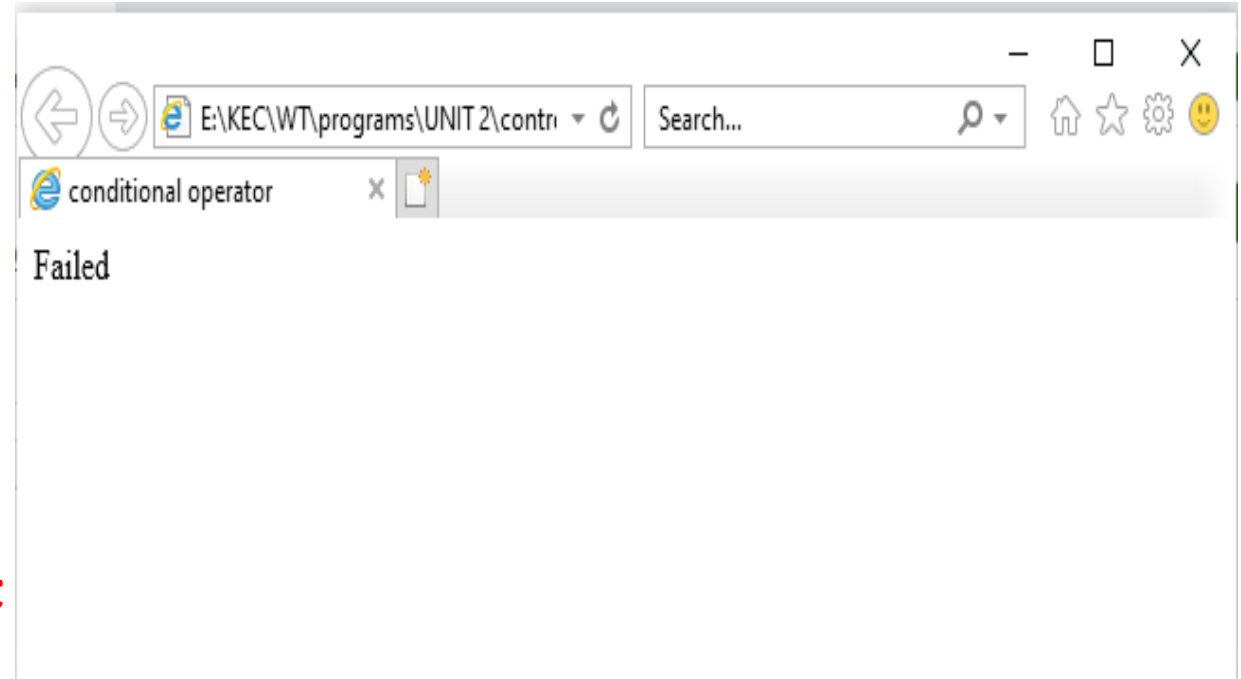
```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>If statement</title>
<script type = "text/javascript">
grade=50;
if ( grade >= 60 )
document.writeln( "<h3>Passed</h3>" );
else
document.writeln( "<h3>failed</h3>" );
</script>
<body>
</body>
</html>
```



failed

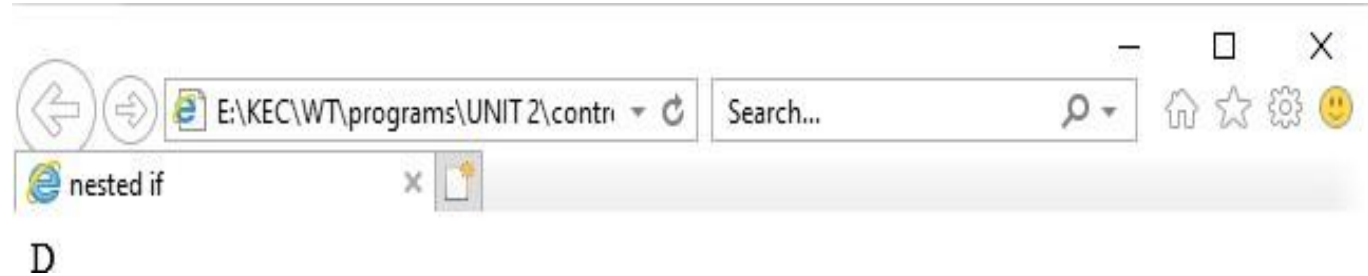
Conditional Operator (?:)-ternary operator

```
<!DOCTYPE html>  
<html>  
<head>  
<meta charset = "utf-8">  
<title>conditional operator</title>  
<script type = "text/javascript">  
    grade=50;  
    document.writeln(grade>=60?"Passed":"Failed");  
</script>  
<body>  
</body>  
</html>
```



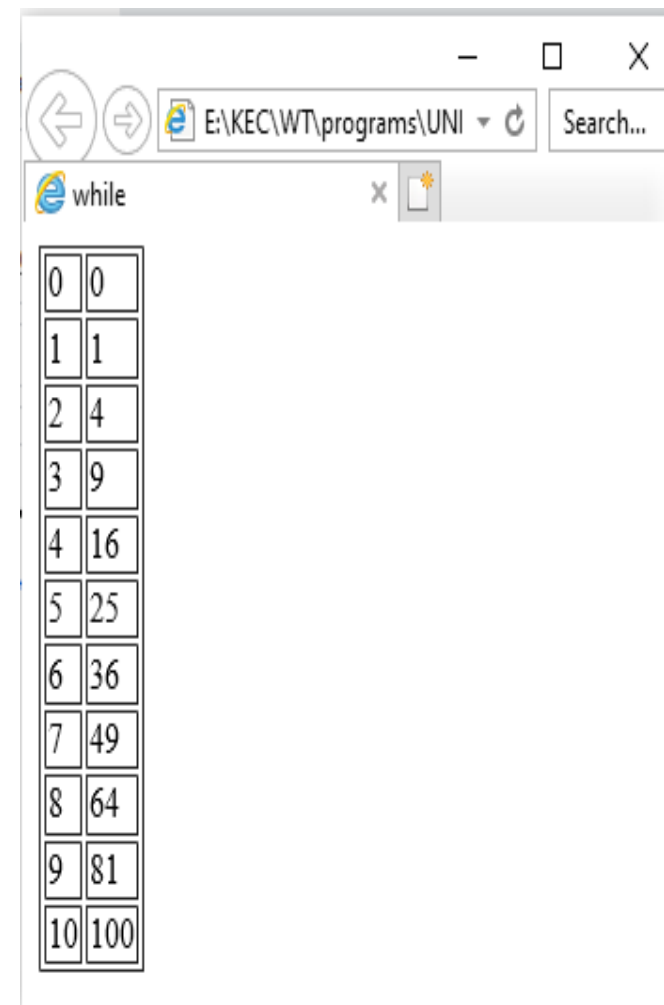
Nested if...else Statements

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>nested if</title>
<script type = "text/javascript">
grade=65;
if (grade >= 90 )
document.writeln( "A" );
else
if (grade >= 80 )
document.writeln( "B" );
else
if (grade >= 70 )
document.writeln( "C" );
else
if (grade >= 60 )
document.writeln( "D" );
else
document.writeln( "F" );
</script>
<body>
</body>
</html>
```



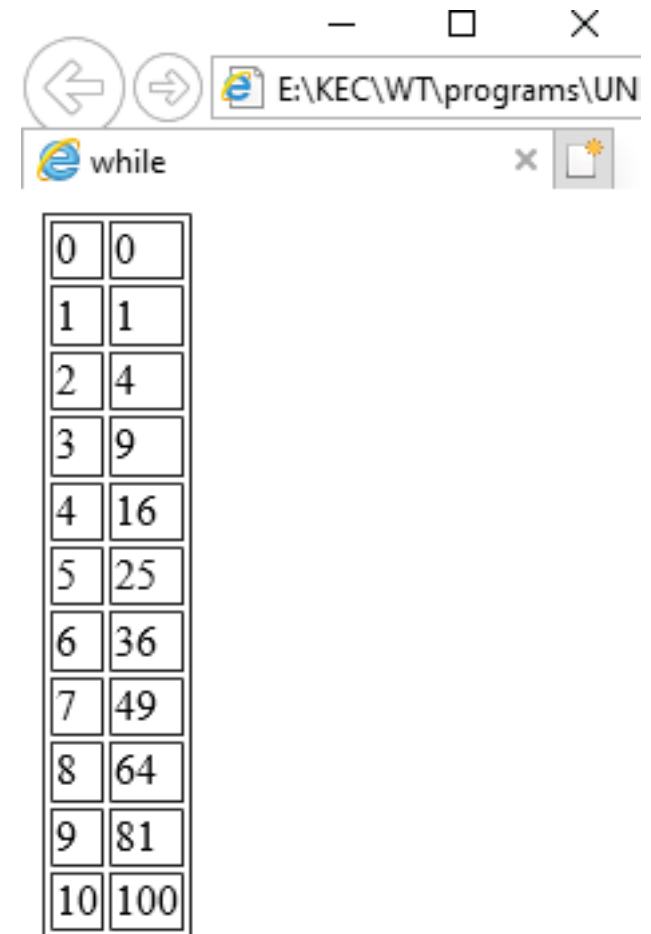
while Repetition Statement

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>while</title>
</head>
<body>
<table border="1">
<script>
var i=0;
while(i<=10)
{
document.write("<tr>"+"<td>" +i+"</td>"+"<td>" +(i*i)+"</td>"+"</tr>");
i++;
}
</script>
</table>
</body>
</html>
```



do...while Repetition Statement

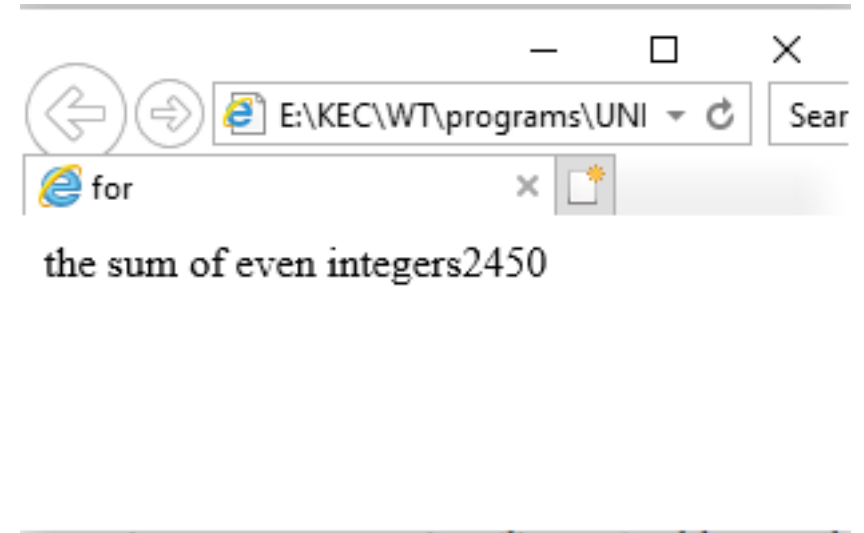
```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>while</title>
</head>
<table border="1">
<script>
var i=0;
do
{
document.write("<tr>"+ "<td>"+i+"</td>"+ "<td>"+(i*i)+"</td>"+ "</tr>");
i++;
}while(i<=10);
</script>
</table>
</body>
</html>
```



0	0
1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100

for Repetition Statement

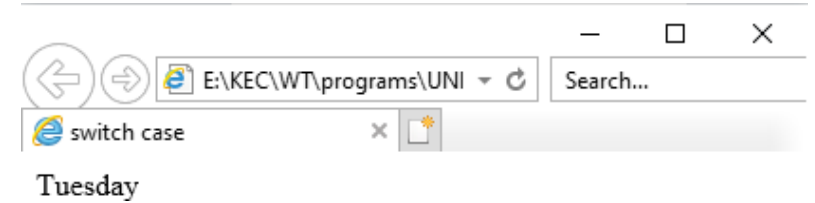
```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>for</title>
<script>
var sum=0;
for(var i=2;i<100;i=i+2)
{
sum=sum+i;
}
document.write("the sum of even integers"+sum);
</script>
<body>
</body>
</html>
```



switch Multiple-Selection Statement

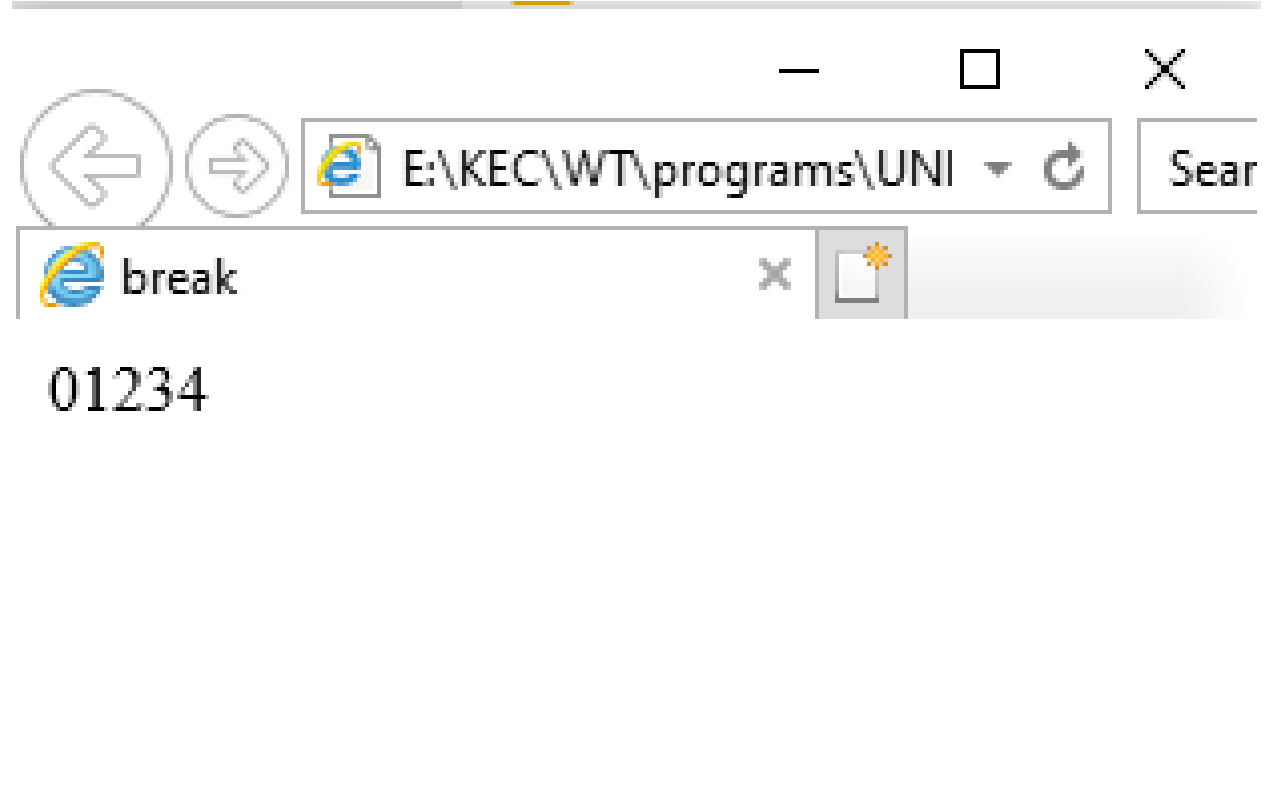
```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>switch case</title>
<script type="text/javascript">
var d=new Date();
var c=d.getDay();
switch(c)
{
case 0:
document.write("Monday");
break;
case 1:
document.write("Tuesday");
break;
case 2:
document.write("wednesday");
break;
case 3:
document.write("thursday");
break;
```

```
case 4:
document.write("friday");
break;
case 5:
document.write("saturday");
break;
case 6:
document.write("sunday");
break;
}
</script>
</head>
<body>
</body>
</html>
```



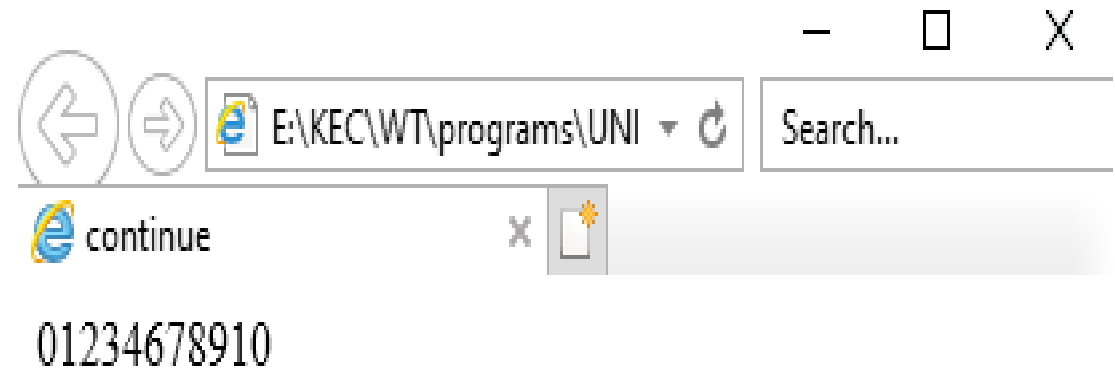
Break Statements

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>break</title>
<script>
for(var i=0;i<=10;i++)
{
if(i==5)
break;
document.write(i);
}
</script>
<body>
</body>
</html>
```



Continue Statements

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>continue</title>
<script>
for(var i=0;i<=10;i++)
{
if(i==5)
continue;
document.write(i);
}
</script>
<body>
</body>
</html>
```



Logical Operators

JavaScript provides logical operators that can be used to form more complex conditions by combining simple conditions. The logical operators are && (logical AND), || (logical OR) and ! (logical NOT, also called logical negation).

logical AND

expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Fig. 8.13 | Truth table for the && (logical AND) operator.

Logical OR:

expression1	expression2	expression1 expression2
false	false	false
false	true	true
true	false	true
true	true	true

Fig. 8.14 | Truth table for the || (logical OR) operator.

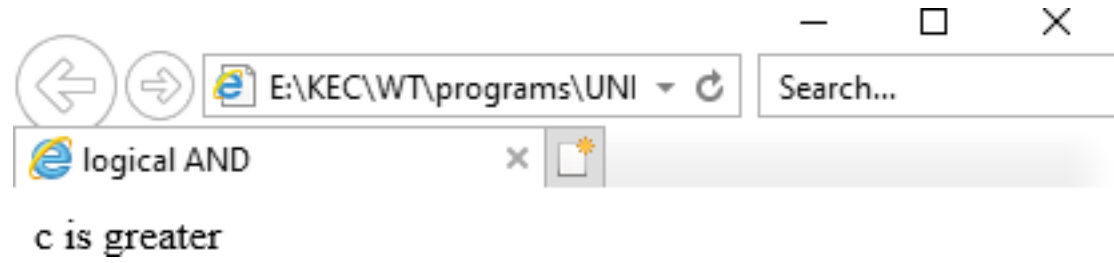
Logical Negation:

expression	!expression
false	true
true	false

Fig. 8.15 | Truth table for operator ! (logical negation).

Logical AND

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>logical AND</title>
<script type="text/javascript">
var a=6,b=5,c=10;
if(a>b && b>c)
document.write("a is greater");
if(b>c)
document.write("b is greater");
else
document.write("c is greater");
</script>
</head>
<body></body>
</html>
```



Logical OR

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset = "utf-8">
```

```
<title>logical OR</title>
```

```
<script type="text/javascript">
```

```
var semavg=90,finalexam=85;
```

```
if(semavg>=90 || finalexam>=90)
```

```
document.write("Grade is A");
```

```
else
```

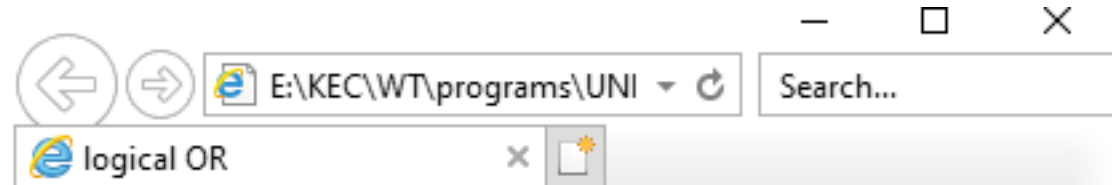
```
document.write("Grade is B");
```

```
</script>
```

```
</head>
```

```
<body></body>
```

```
</html>
```



Grade is A

Java script Functions

- A JavaScript function is a **block of code designed to perform a particular task.**
- A JavaScript function is executed when "something" invokes it (calls it).
- Experience has shown that the best way to develop and maintain a large program is to construct it from small, simple pieces, or modules.
- This technique is called divide and conquer.
- A **function is invoked (that is, made to perform its designated task) by a function call.**
- **The function call specifies the function name and provides information (as arguments) that the called function needs to perform its task.**

Functions

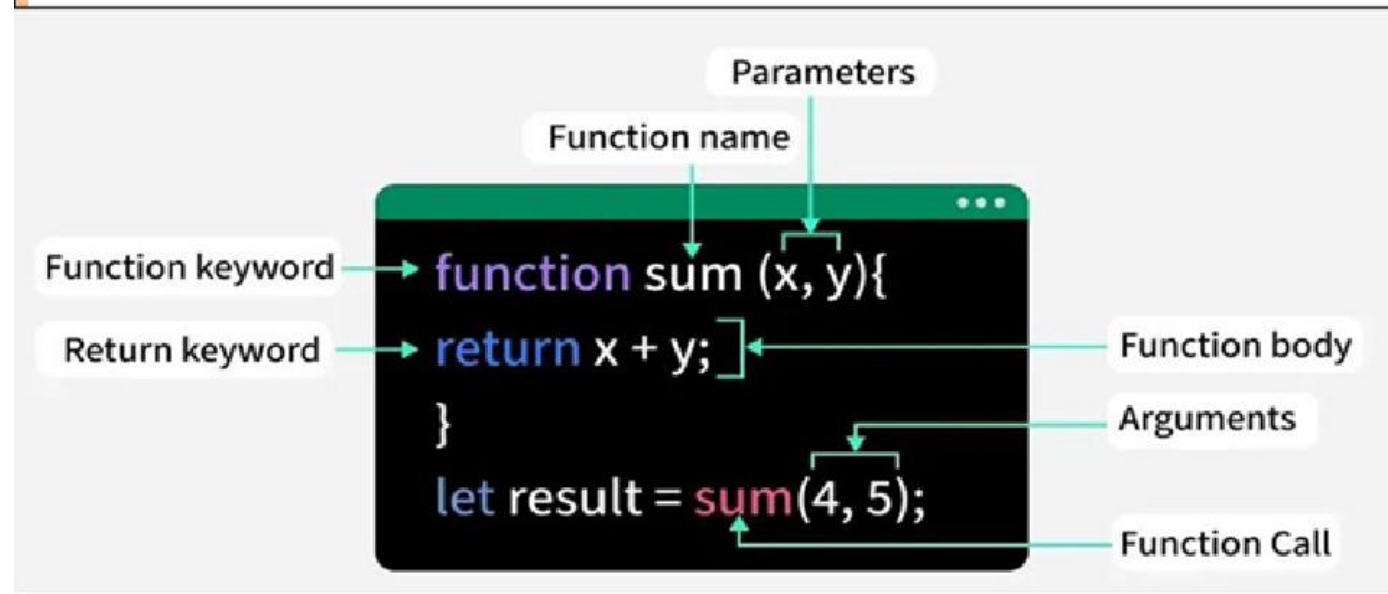
A function is a reusable block of code designed to perform a specific task.

Functions are first-class objects, meaning they can be:

- ✓ ***Assigned to variables***
- ✓ ***Passed as arguments to other functions***
- ✓ ***Returned from other functions***



1. Function Declaration



```
function fun_Name() (parameters)  
{  
  // function body  
  return value;  
}  
fun_Name();
```

```
<script>  
function add(a, b) {  
  return a + b;  
}  
var result = add(2, 3);  
document.write(result);  
</script>
```

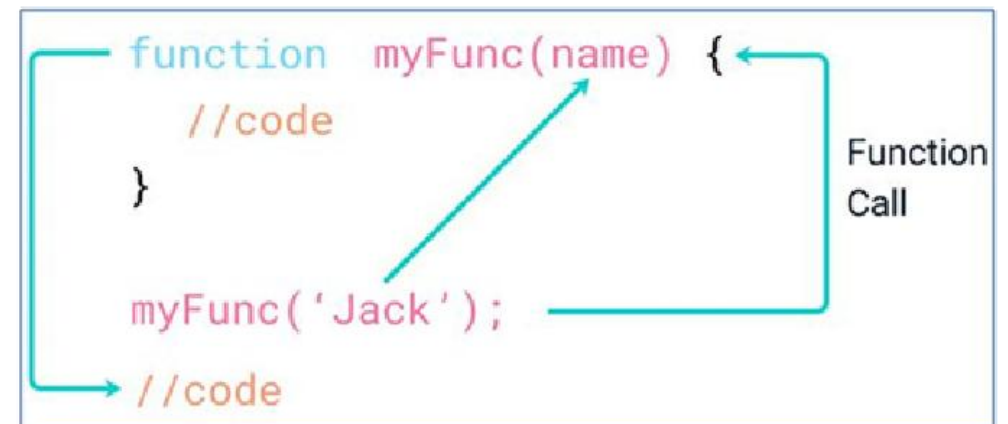
Function Parameters and Arguments

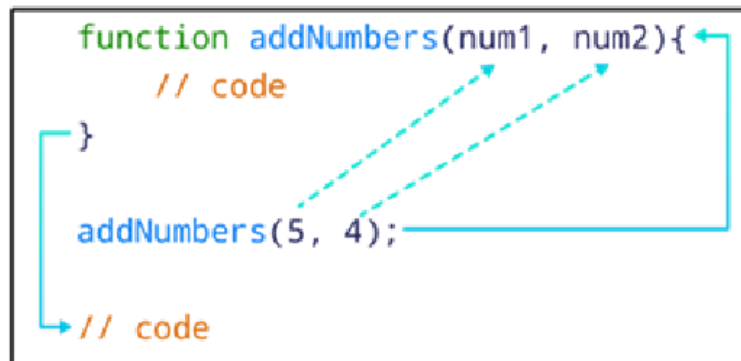
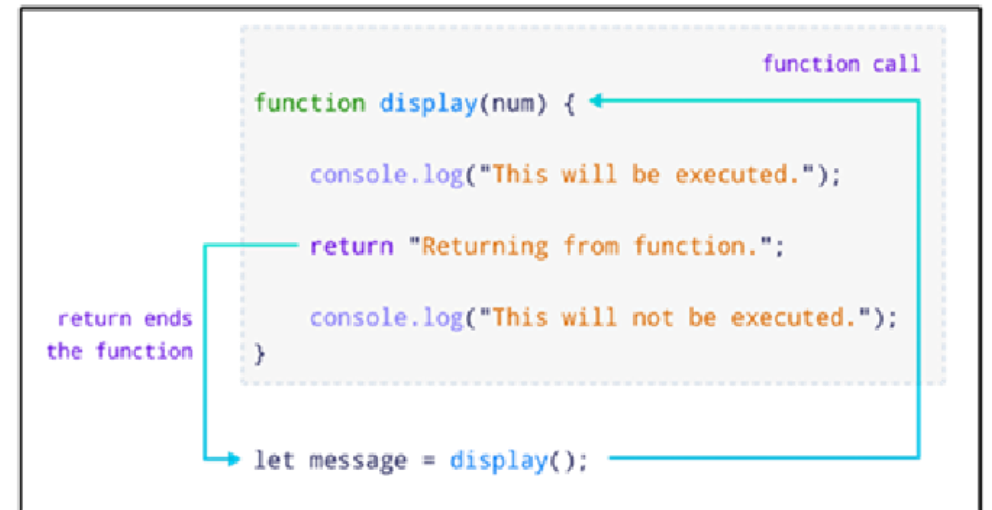
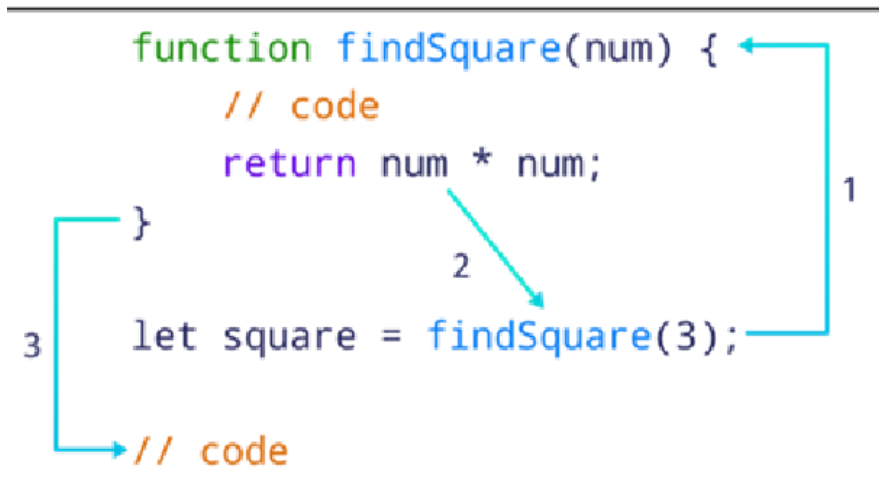
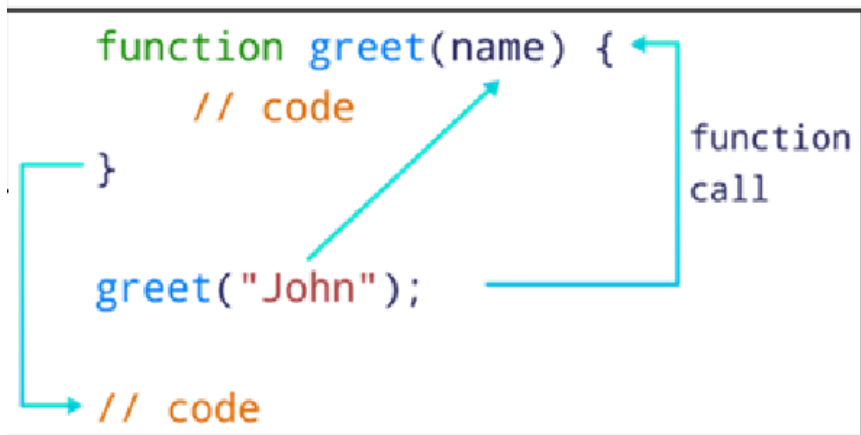
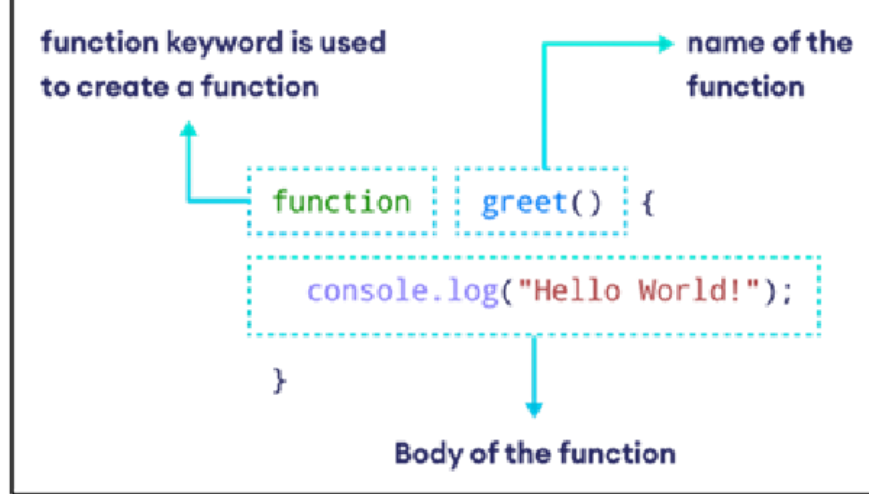
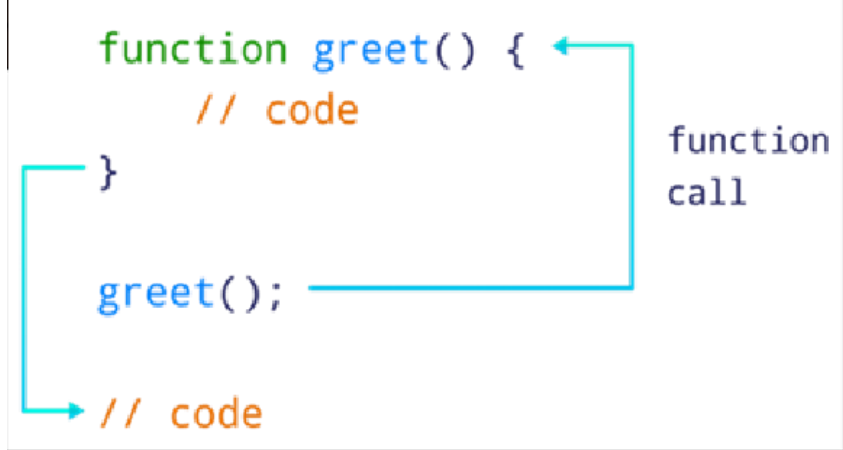
- Parameters are variables listed in the function definition.
- Arguments are actual values passed to the function when it is called.

Return Statement

The return statement:

- ✓ Ends function execution
- ✓ Sends a value back to the caller





2. Function Expression

- A function expression assigns a function to a variable.

```
<!DOCTYPE html>
<html lang="en">
<body>
  <h1>Function Expression</h1>
  <script>
    const greet = function ()
    {
      console.log("Hello");
      document.write("Hello");
    };
    greet();
  </script>
</body> </html>
```

The screenshot shows a web browser window with the title "Function Expression". The page content displays the text "Hello" in a red box. The browser's developer tools are open, showing the "Elements" panel on the right. The "script" tab is selected, displaying the following code:

```
<!DOCTYPE html>
<html lang="en">
<head></head>
<body>
  <h1>Function Expression</h1>
  <script> -- $0
    const greet = function () {
      console.log("Hello");
      document.write("Hello");
    };
    greet();
  </script>
```

Below the code, the "Console" panel is open, showing a single log entry: "Hello".

3. Arrow functions

- Arrow functions is a *short method* for writing a function expression.

```
function Add(num1, num2) {  
  return num1 + num2;  
}
```

↑
Normal Function

```
var Add = (num1, num2) => {  
  return (num1+num2)  
}
```

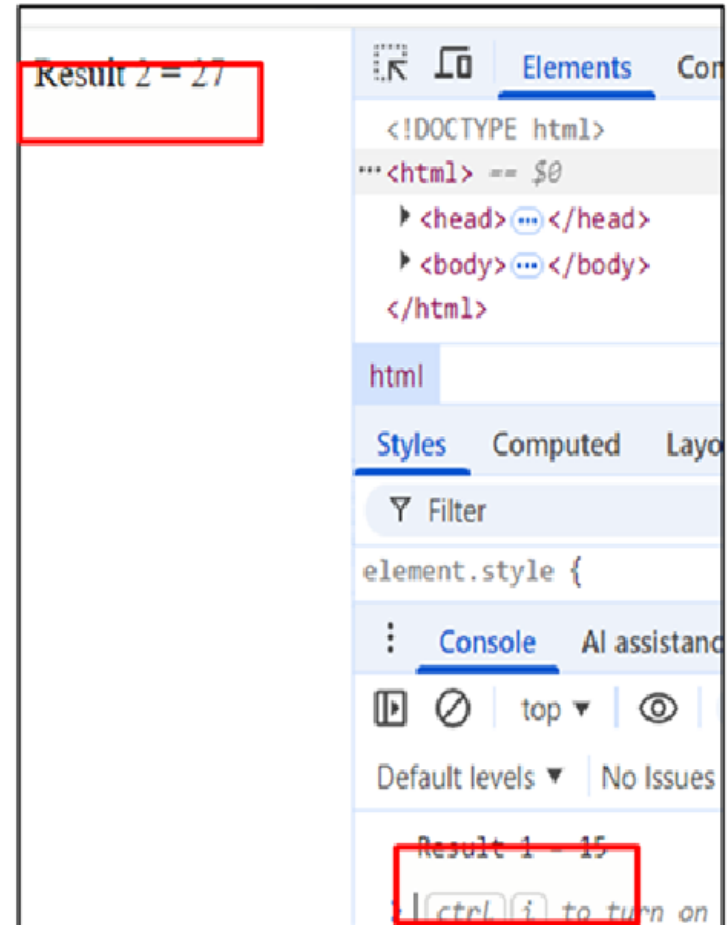
↑
Arrow Function

() => { }

```
const add = function(a, b)  
{  
  return a + b;  
};
```

```
const add = (a, b) =>  
{  
  return a + b;  
};
```

```
<script>  
const add = (a, b) => a + b;  
  const result1 = add(5, 10);  
  const result2 = add(20, 7);  
  console.log("Result 1 = " + result1);  
  document.write("Result 2 = " + result2);  
</script>
```



Riya is a student in school She received her exam marks:

- Math: 80
- Science: 70
- English: 90

Her teacher wants to quickly calculate her **total marks** and show it on a webpage.



```
<html> <body>
<script>
  function totalMarks(a, b, c) {
    return a + b + c;
  }
  var math = 80;
  var science = 70;
  var english = 90;

  var total = totalMarks(math, science, english);

  document.write("<h3>Riya's Exam Result</h3>");
  document.write("Math: " + math + "<br>");
  document.write("Science: " + science + "<br>");
  document.write("English: " + english + "<br>");
  document.write("<b>Total Marks: " + total + "</b>");
</script> </body> </html>
```

Riya's Exam Result

Math: 80

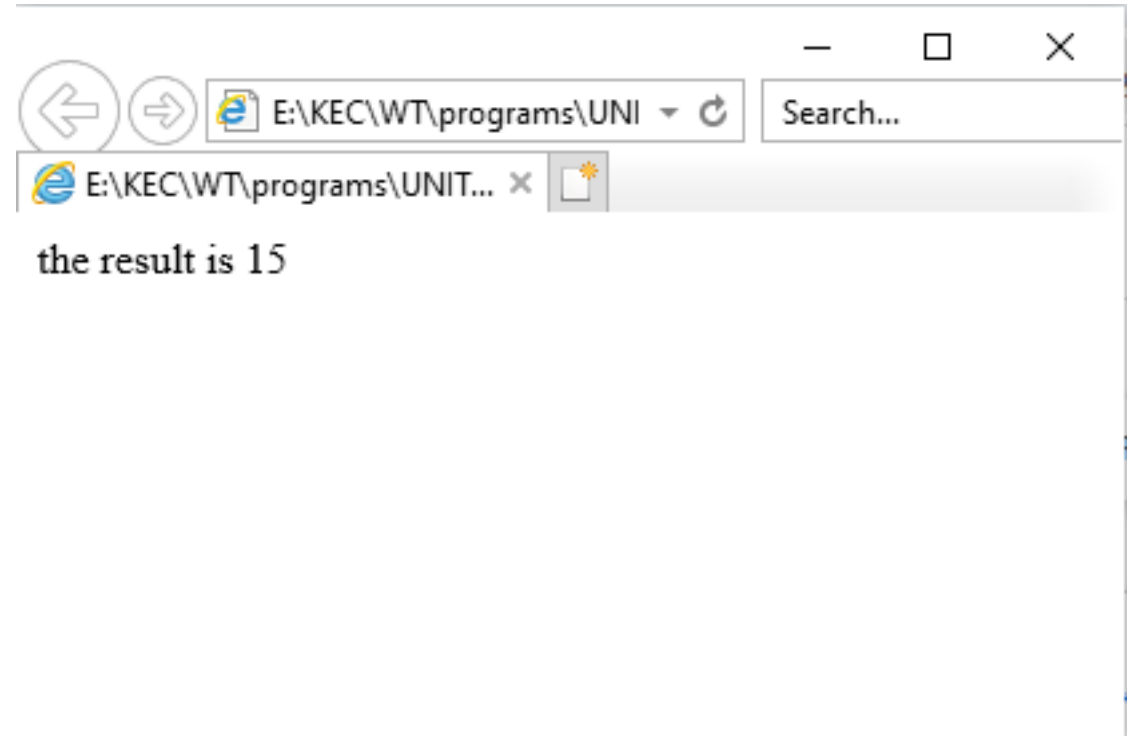
Science: 70

English: 90

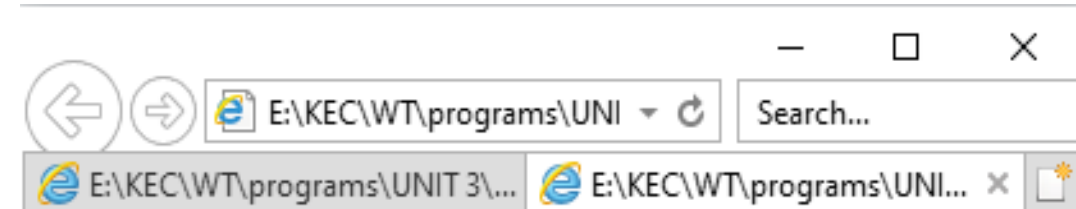
Total Marks: 240

Function Definitions

```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript">
var a=10,b=5;
function add(a,b)
{
c=a+b;
document.write("the result is "+c);
}
add(a,b);
</script>
</head>
<body>
</body>
</html>
```



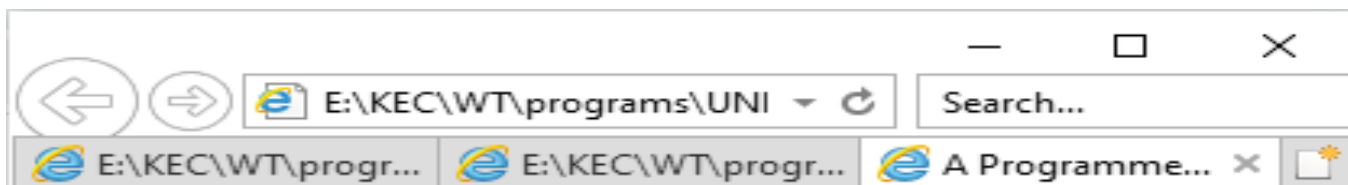
```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript">
var a=10,b=20;
function add(a,b)
{
c=a+b;
return c;
}
document.write("the result is"+add(a,b));
</script>
</head>
<body>
</body>
</html>
```



the result is30

Programmer-Defined Function square

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>A Programmer-Defined square Function</title>
<script>
function square( y )
{
return y * y;
}
document.writeln( "<h1>Square the numbers from 1 to 10</h1>" );
for ( var i = 0; i <= 10; i++ )
document.writeln( "<p>The square of " + i + " is " +square( i )+ "</p>" );
</script>
</head><body></body></html>
```



Square the numbers from 1 to 10

The square of 0 is 0

The square of 1 is 1

The square of 2 is 4

The square of 3 is 9

The square of 4 is 16

The square of 5 is 25

The square of 6 is 36

The square of 7 is 49

The square of 8 is 64

The square of 9 is 81

The square of 10 is 100

Programmer-Defined Function maximum

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>Maximum of Three Values</title>
<script>
var input1 = window.prompt( "Enter first number");
var input2 = window.prompt( "Enter second number");
var input3 = window.prompt( "Enter third number");
var value1 = parseInt( input1 );
var value2 = parseInt( input2 );
var value3 = parseInt( input3 );
var maxValue = maximum( value1, value2, value3 );
document.writeln( "Maximum is: " + maxValue);
function maximum( x, y, z )
{
return Math.max( x, Math.max( y, z ) );
}
</script>
</head><body></body>
</html>
```

Explorer User Prompt

Script Prompt:
Enter first number

10

OK
Cancel

Explorer User Prompt

Script Prompt:
Enter second number

20

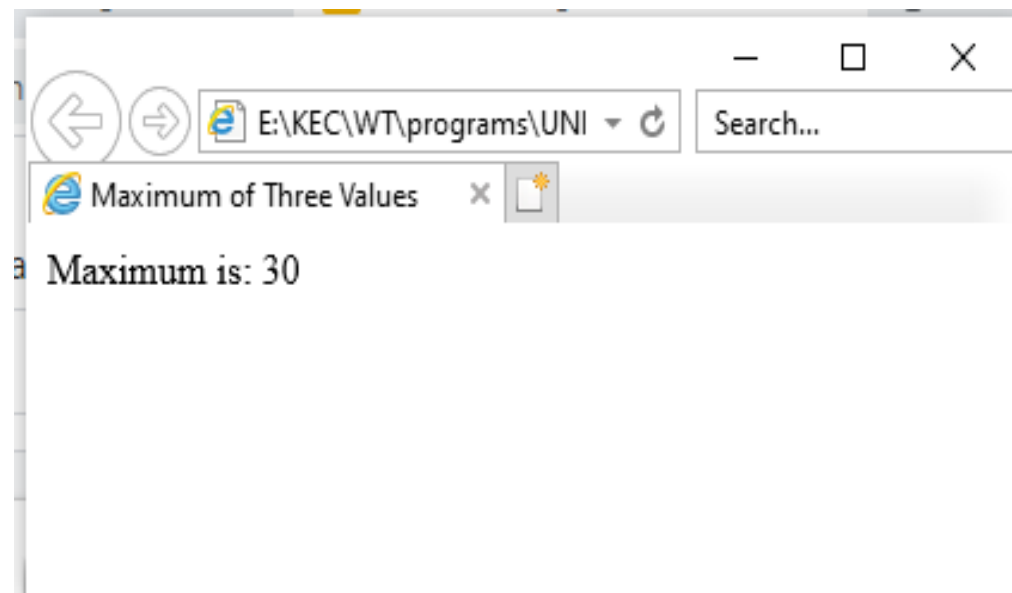
OK
Cancel

Explorer User Prompt

Script Prompt:
Enter third number

30

OK
Cancel



Java Script Arrays

- **Arrays are data structures** consisting of related data items
- **An array is a group of memory locations that all have the same name and normally are of the same type**
- **To refer to a particular location** or element in the array, we **specify the name of the array and the position number** of the particular element in the array.
- JavaScript arrays are used to store multiple values in a single variable.
- An array is a special variable, which can hold more than one value at a time and you can access the values by referring to an **index number**.

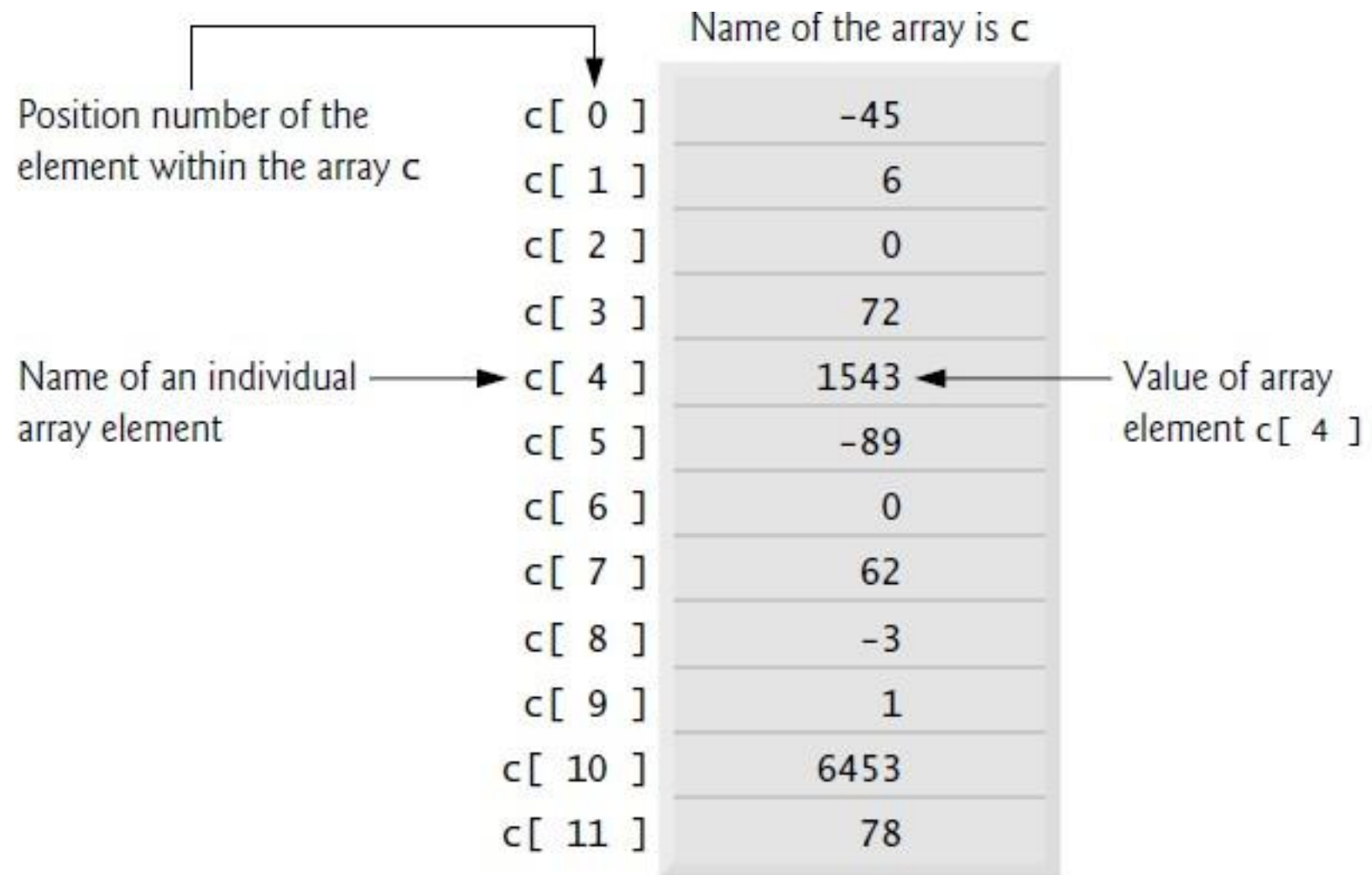


Fig. 10.1 | Array with 12 elements.

Declaring and Allocating Arrays

- Arrays occupy space in memory.
- Actually, an array in JavaScript is an Array object.
- You use the new operator to create an array and to specify the number of elements in an array.
- The new operator creates an object as the script executes by obtaining enough memory to store an object of the type specified to the right of new.
- To allocate 12 elements for integer array c, use a new expression like:

```
var c = new Array( 12 );
```

- The preceding statement can also be performed in two steps, as follows:

```
var c; // declares a variable that will hold the array
```

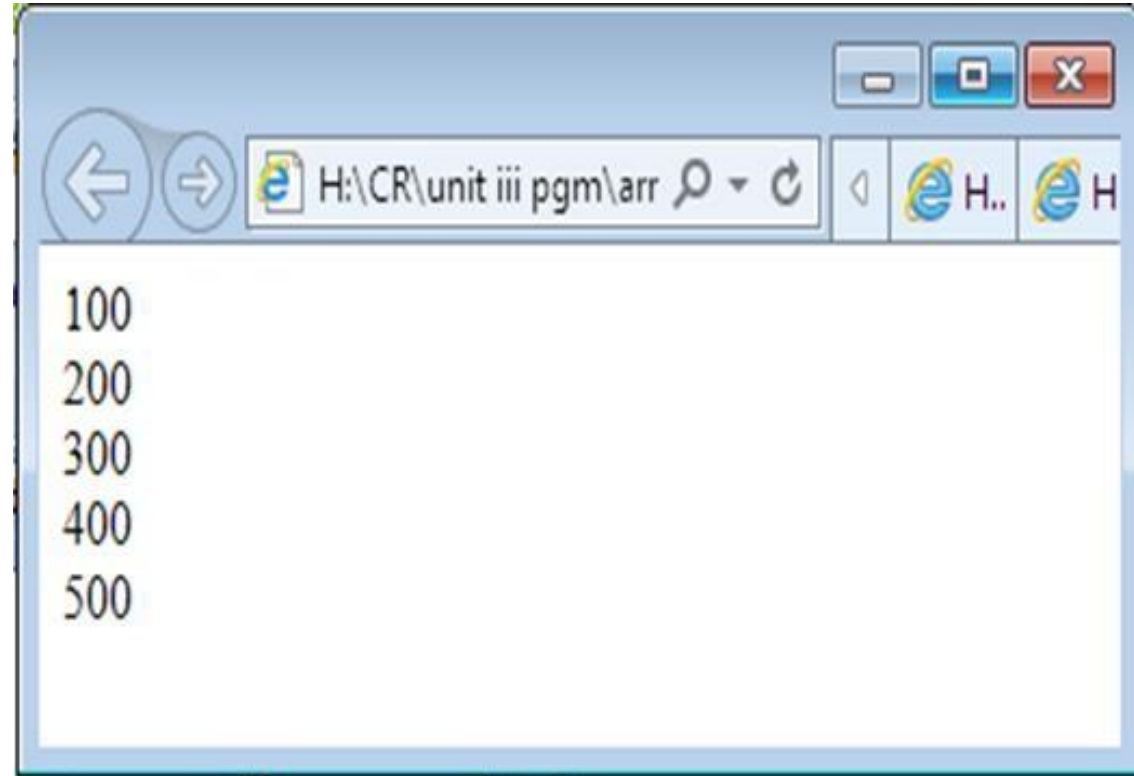
```
c = new Array( 12 ); // allocates the array
```

- When arrays are created, the elements are not initialized—they have the value undefined.

Examples Using Arrays

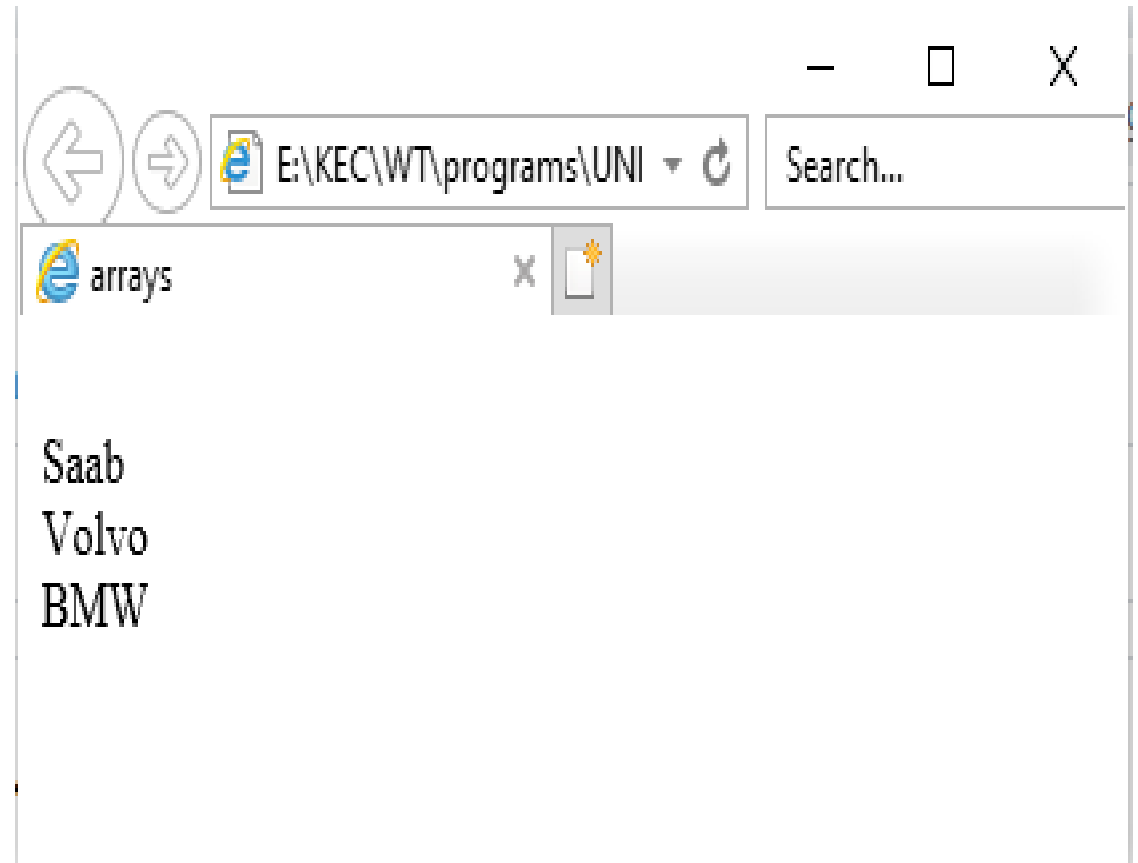
One dimensional array

```
<!DOCTYPE html>
<html>
<body>
<script>
var a=new Array(5);
a=[100,200,300,400,500];
for(i=0;i<5;i++)
{
document.write(a[i]);
document.write("<br>");
}
</script>
</body>
</html>
```



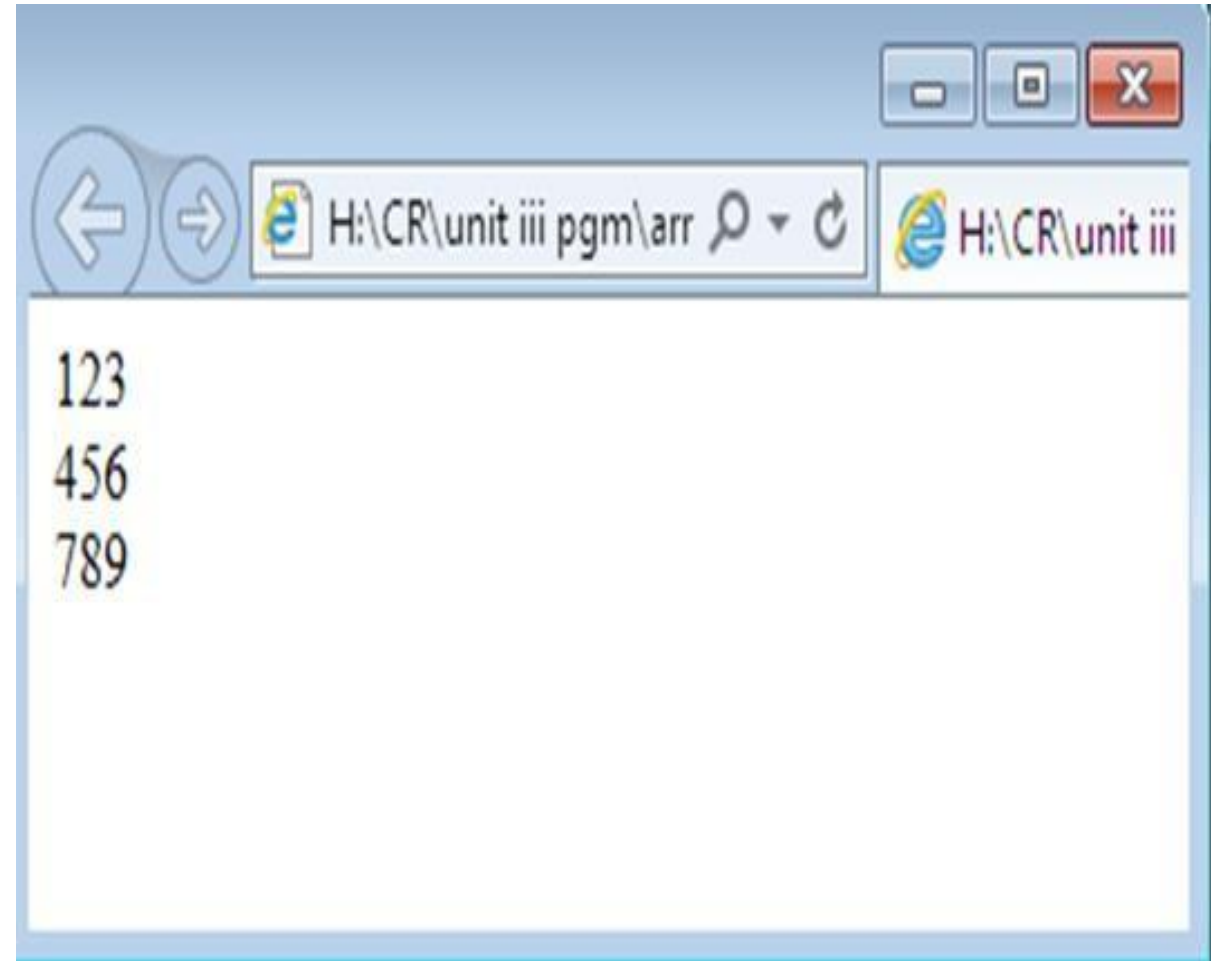
Examples Using Arrays

```
<!DOCTYPE html>
<html>
<head>
<title> arrays</title>
<meta charset="UTF-8">
<script type="text/javascript">
var cars = ["Saab", "Volvo", "BMW"];
for(i=0;i<3;i++)
{
document.write("<br>" + cars[i]);
}
</script>
<body>
</body>
</html>
```



Two dimensional array

```
<!DOCTYPE html>
<html>
<body>
<script>
var a=new Array();
a=[[1,2,3],[4,5,6],[7,8,9]];
for(i=0;i<3;i++)
{
for(j=0;j<3;j++)
{
document.write(a[i][j]);
}
document.write("<br>");
}
</script>
</body>
</html>
```



Indirectly embedding of javascript

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> arrays</title>
```

```
<meta charset="UTF-8">
```

```
<script type="text/javascript" src="a.js">
```

```
</script>
```

```
<body>
```

```
</body>
```

```
</html>
```

a.js

```
function myfun()//function definition
{
  var cars = ["Saab", "Volvo", "BMW"];
  for(i=0;i<3;i++)
  {
    document.write("<br>" + cars[i]);
  }
}
myfun();//function call
```



E:\KEC\WT\programs\UNI ▾



Search...



arrays



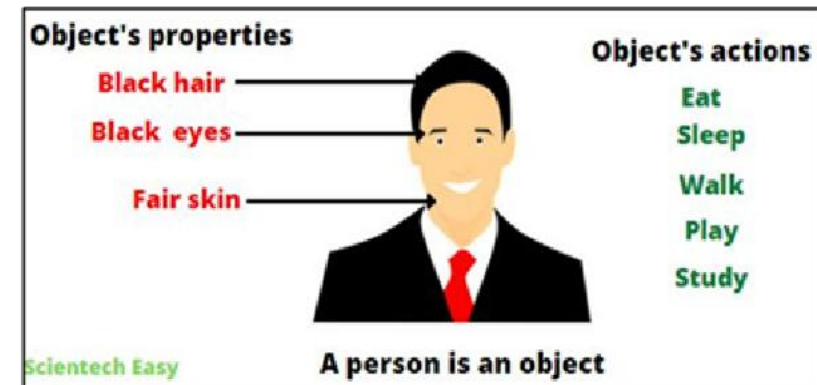
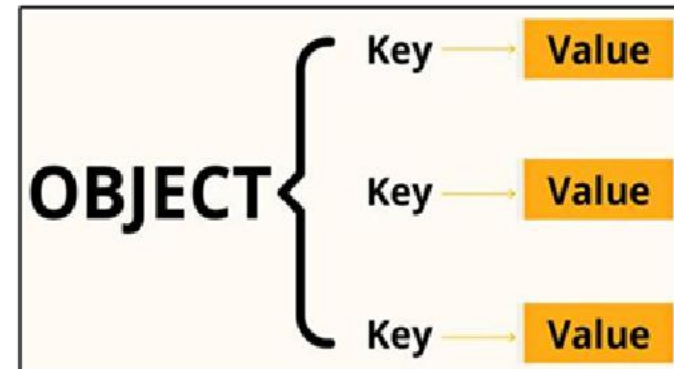
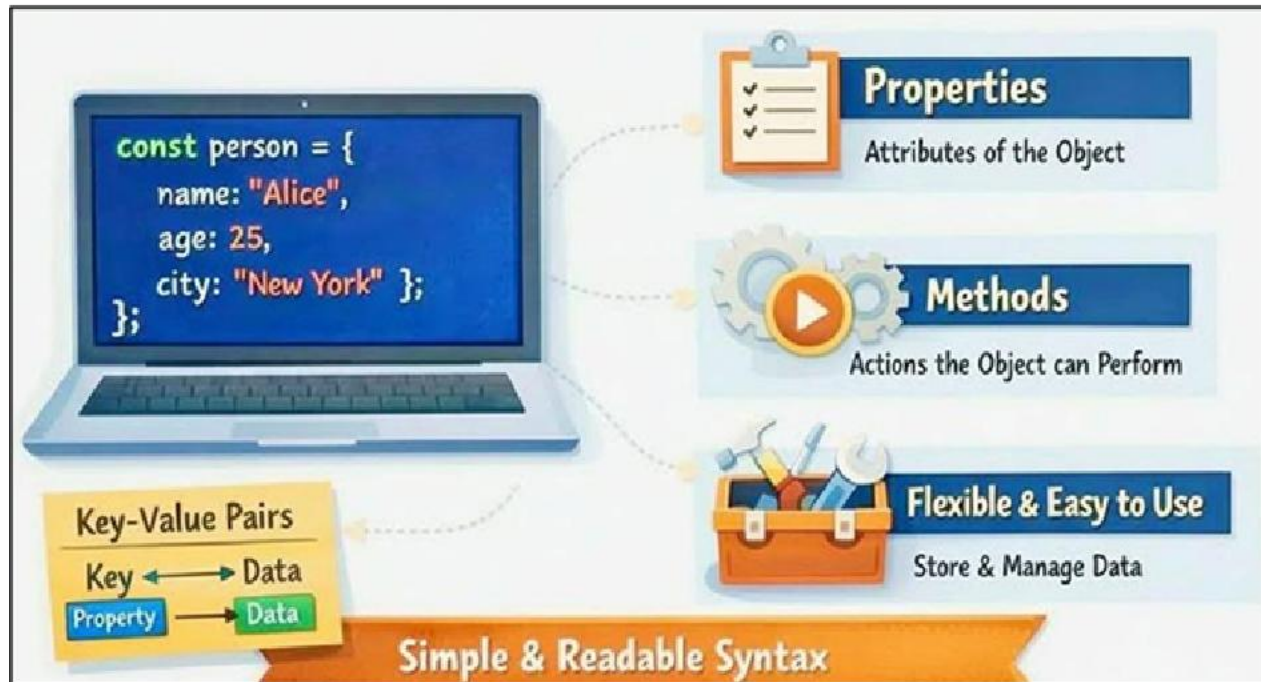
arrays



Saab
Volvo
BMW

OBJECTS

- JavaScript object is a variable that can store multiple data in **key-value** pairs.
- An object is an **unordered collection of properties**, where:
 - **Key (property name)** → string or Symbol
 - **Value** → any JS value (primitive, function, array, another object, etc.)



Creation Using Object Literal

- The object literal syntax allows you to define and initialize an object with curly braces {}, setting properties as key-value pairs.

```
const objectName = {  
  key1: value1,  
  key2: value2,  
  ...,  
  keyN: valueN  
};
```

Creation Using new Object() Constructor

- Objects created with new Object() start empty and are typically populated imperatively.

```
const user = new Object();  
  
user.name = "Ravi";  
user.age = 30;  
user.isAdmin = false;  
  
console.log(user);
```

Object Operations

1. Modify Object Properties

- Object properties can be modified by assigning a new value to an existing key.

```
const person = {  
  name: "Bai ",  
  hobby: "Reading",  
};  
  
person.hobby = "Singing";  
  
console.log(person);
```

```
▶ {name: 'Bai ', hobby: 'Singing'}
```

2. Add Object Properties

```
const student = {  
  name: "John",  
  age: 20,  
};  
  
student.rollNo = 14;  
student.faculty = "Science";  
  
console.log(student);
```

```
{ name: 'John', age: 20, rollNo: 14,  
  faculty: 'Science' }
```

3. Delete Object Properties

- Remove properties from an object using the delete operator.

```
const employee = {  
  name: "Tony",  
  position: "Officer",  
  salary: 30000,  
};  
  
delete employee.salary  
  
console.log(employee);
```

```
{name: 'Tony', position: 'Officer' }
```

Access Object Properties

1. Dot property accessor

- A common way to access the property of an object is the *dot property accessor*.
- Use a dot when you **know the property name**.

```
const car = {  
  brand: "Toyota",  
  model: "Corolla",  
  year: 2022  
};  
console.log(car.brand);  
console.log(car.year);
```

2. Square brackets property accessor

- Read or write object properties using an expression instead of a literal identifier.

expression[expression]

- Use brackets when:
 - ***The property name is dynamic***
 - ***The property has spaces***
 - ***The property name is stored in a variable***

```
const user = {  
  name: "Alex",  
  "favorite color": "blue"  
};  
console.log(user["name"]);  
console.log(user["favorite color"]);  
  
const key = "age";  
user[key] = 25;  
console.log(user["age"]);
```

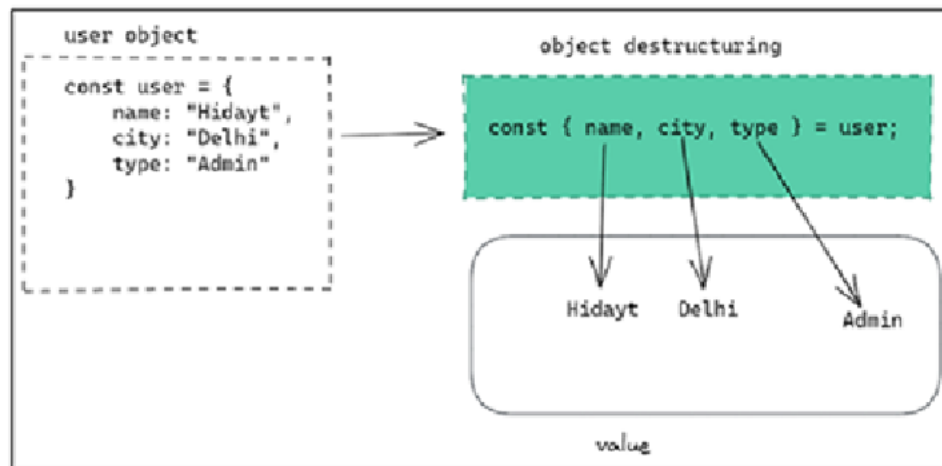
3. Nested Object Properties

- Objects inside objects

```
const user = {  
  name: "Bai",  
  address: {  
    city: "New York",  
    zip: 10001  
  }  
};  
console.log(user.address.city);
```

4. Destructuring

- Shortcut to extract properties from an object and store them into variables



```
var user = {  
  name: "Hidayt",  
  city: "Delhi",  
  type: "Admin"  
}  
  
var { name, city, type } = user;  
  
console.log(name);  
console.log(city);  
console.log(type);
```

Javascript object

- Everything in JavaScript takes the form of an ***object***.
- ***This means that it is a "package", which contains various properties (pieces of information) and methods (lists of instructions that it can carry out).***

DOM

- **DOM - Document Object Model**
- When a web page is loaded, the browser creates a **Document Object Model** of the page.
- *The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document.*
- The **HTML DOM** model is constructed as a tree of **Objects**:

The HTML DOM is a standard **object** model and **programming interface** for HTML.

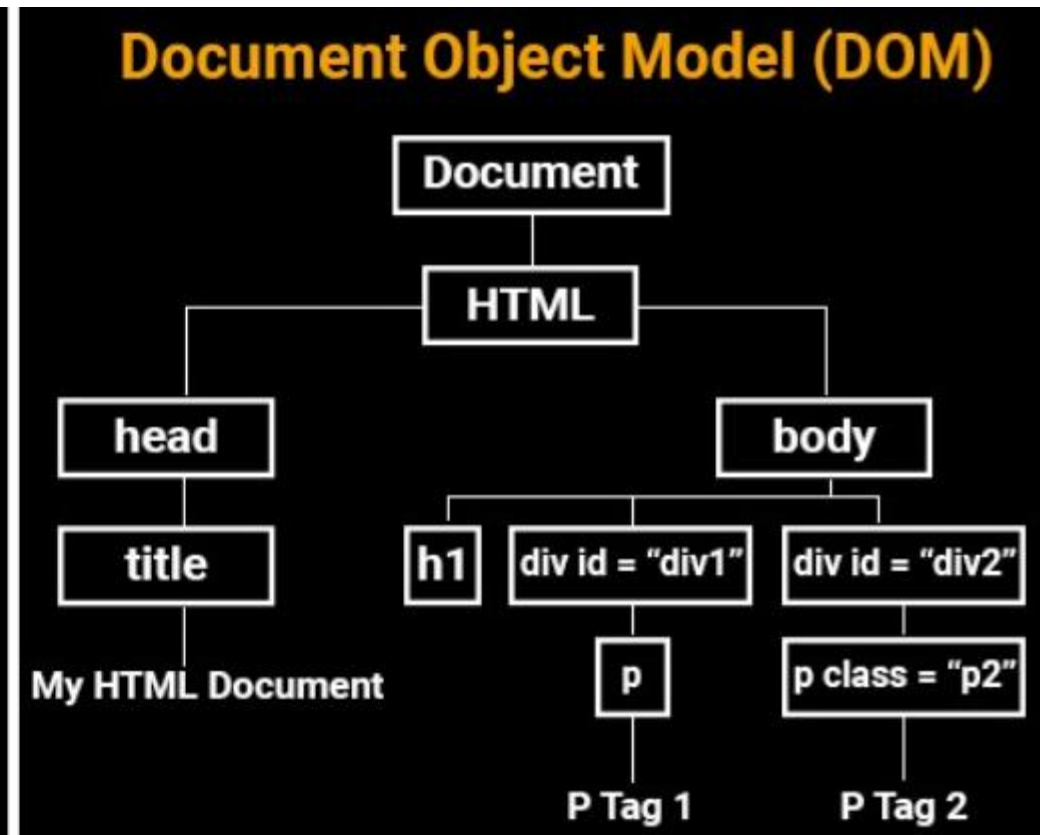
It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

- The Document Object Model (DOM) defines the logical structure of documents and the way a document is accessed and manipulated.

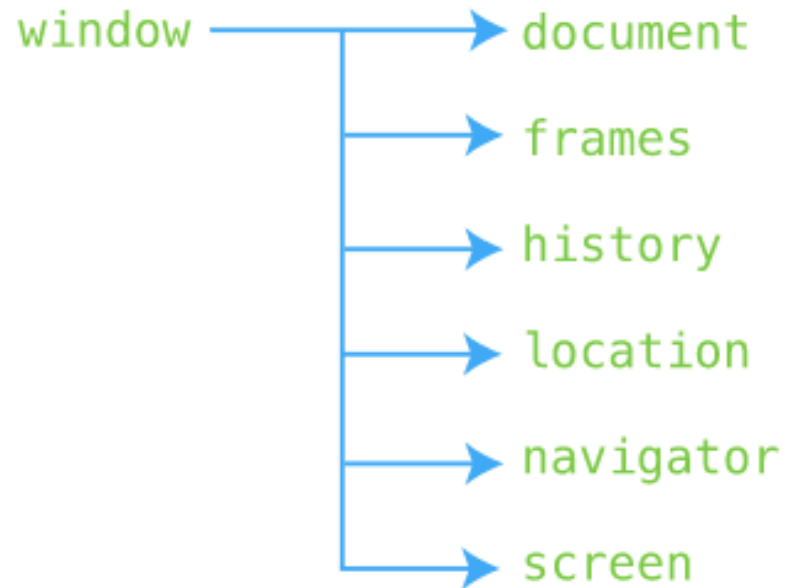
HTML Document

```
index.html x
1 <html>
2   <head>
3     <title>My HTML Document</title>
4   </head>
5
6   <body>
7     <h1>Heading</h1>
8     <div id="div1">
9       <p>P Tag 1</p>
10    </div>
11    <div id="div2">
12      <p class="p2">P Tag 2</p>
13    </div>
14  </body>
15 </html>
```



Browser Object Model

- The Browser Object Model (BOM) allows JavaScript to communicate to the browser.



Window Object

- The represents the browser's window.
- All global JavaScript objects, functions, and variables automatically become members of the window object.
- Even the document object is a property of the window object.

`window.document.getElementById("pg1");`

is the same as:

`document.getElementById("pg1");`

- Properties:
 - window.innerHeight - the inner height of the browser window (in pixels)
 - window.innerWidth - the inner width of the browser window (in pixels)
- methods

<code>alert()</code>	Opens a popup window. User must click OK to close.
<code>prompt()</code>	Prompts user for an input
<code>open()</code>	Opens new browser window with specified URL.
<code>close()</code>	Closes window.
<code>print()</code>	Opens print dialog.

Moving a window object

There are also functions you can use to move the window object.

<code>moveBy()</code>	Move the window relative.
<code>moveTo()</code>	Moves the window to a specific location.
<code>resizeBy()</code>	Change size of window relative to current size.
<code>resizeTo()</code>	Change size of window to specified size.

The document object

- The first object that you will meet - document.
- This refers to the main screen in the browser itself, the part where the text and images appear.
- The document has a property (among others) called `bgColor`, which specifies what the background color of the screen

- bgColor is an example of a document *property*.
- *An example of its methods is write().*

```
<SCRIPT LANGUAGE="JavaScript">  
    document.bgColor = "YELLOW";  
    document.write("This should appear on a yellow  
background.");  
    document.write("<P>");  
    document.write("This is my second JavaScript  
program.");  
    document.write("</P>");  
</SCRIPT>
```

example

```
<SCRIPT LANGUAGE="JavaScript">  
document.write("<IMG SRC=my_pic.gif>");  
document.write("<HR WIDTH=40%>");  
document.write("<FONT COLOR=#4050ff> hello</font>");  
  
</SCRIPT>
```

Program

```
<HTML>
```

```
<BODY>
```

```
<SCRIPT LANGUAGE="JavaScript">
```

```
document.bgColor = "YELLOW";
```

```
</SCRIPT>
```

You can add some more text and/or HTML tags here, of course.

```
</BODY>
```

```
</HTML>
```


Properties	Description
cookie	returns a report that contains all the visible and unexpired cookies associated with the document
domain	returns the domain name of the server from which the document has originated
lastModified	returns the date on which document was last modified
documentMode	returns the mode used by the browser to process the document
readyState	returns the loading status of the document.
referrer	returns the URL of the documents referred to in an HTML document
title	returns the name of the HTML document defined between the starting and ending tags of the TITLE element
URL	returns the full URL of the HTML document.

```
<!doctype html>
  <head>
    <title>Document Properties</title>
  </head>
  <body>
    <h3>Document Properties Example</h3>

    <script>

      document.write(document.domain + "<br>")
      document.write(document.lastModified + "<br>")
      document.write(document.title + "<br>")
      document.write(document.url + "<br>")

    </script>
  </body>
</html>
```

Methods	Description	Syntax
<code>open()</code>	opens an HTML document to display the output	<code>document.open(mimetype, replace)</code>
<code>close()</code>	closes an HTML document	<code>document.close()</code>
<code>write()</code>	Writes HTML expressions or JavaScript code into an HTML document	<code>document.write(exp1, exp2, ...)</code>
<code>writeln()</code>	write a new line character after each HTML expressions or JavaScript Code	<code>document.writeln(exp1, exp2, ...)</code>
<code>getElementById()</code>	returns the reference of first element of an HTML document with the specified id.	<code>document.getElementById(id)</code>
<code>getElementByName()</code>	returns the reference of an element with the specified name	<code>document.getElementsByName(name)</code>

```
<html>
<body>

<h1>My Web Page</h1>

<p id="myPar">I am a paragraph.</p>
<div id="myDiv">I am a div.</div>

<p>
<button type="button" onclick="myFunction()">Try it</button>
</p>

<script>
function myFunction() {
    document.getElementById("myPar").innerHTML = "Hello Dolly.";
    document.getElementById("myDiv").innerHTML = "How are you?";
}
</script>

<p>When you click on "Try it", the two elements will change.</p>

</body>
</html>
```

Screen Object

- JavaScript Screen is a built-in Interface (object type) that is used to fetch information related to the browser screen on which the current window is rendered.
- It provides information about the dimensions of the display screen such as its height, width, color bits, etc.

JavaScript Screen Object Properties

JavaScript Screen object includes some properties that are used to get browser screen information. A table of such properties is given below.

Property	Description
availHeight	specifies the height of screen, excluding the windows taskbar
availWidth	specifies the width of the screen, excluding the Windows taskbar
colorDepth	specifies the depth of color palette, in bits, to display images
height	specifies the total height of the screen
pixelDepth	specifies the color Resolution, in bits per pixel, of the screen
width	specifies the total width of the screen

```
<!DOCTYPE HTML>
<body>
  <script Language="javascript">
    document.writeln("Total Height : " +screen.height+"<br/>");
    document.writeln("Total Width : " +screen.width+"<br/>");
    document.writeln("Available Width : " +screen.availWidth + "<br/>");
    document.writeln("Available Height : " +screen.availHeight + "<br/>");
    document.writeln("Screen Color Depth : " +screen.colorDepth + "<br/>");
    document.writeln("Screen Pixel Depth : " + screen.pixelDepth + "<br/>");
  </script>
</body>
</html>
```

-

- The Screen object colorDepth property is the number of bits used to represent the color of a single pixel.
- It returns the bit depth of the color palette for displaying images.

JavaScript Screen Object Methods

- There are no direct methods in the Screen method object.

Uses of Screen Object

- The JavaScript Screen object can be used for improving the user experience by controlling the UI of the web app or website based on the screen size.

Location Object

- The `window.location` object can be used to get the current page address (URL) and to redirect the browser to a new page.
 - `window.location.href` returns the href (URL) of the current page
 - `window.location.hostname` returns the domain name of the web host
 - `window.location.pathname` returns the path and filename of the current page
 - `window.location.protocol` returns the web protocol used (http: or https:)
 - `window.location.assign()` loads a new document
 - `window.location.reload()` Reload current document.
 - `window.location.replace()` Replace current document.

History Object

- The window.history object contains the browsers history.
 - `history.back()` - same as clicking back in the browser
 - `history.forward()` - same as clicking forward in the browser
 - `history.go()` - Loads specific URL from history list.

```
<html>
<head>
<script>
function goBack()
{
  window.history.back() //history.back();
}
</script>
</head>
<body>

<input type="button" value="Back" onclick="goBack()">

</body>
</html>
```

Javascript Objects

- String
- Date
- Number
- Math
- Array
- RegExp

1.Date Object

- The Date object is used to work with dates and times.
- Date objects are created with new Date().

There are four ways of instantiating a date:

```
var d = new Date();  
var d = new Date(milliseconds);  
var d = new Date(dateString);  
var d = new Date(year, month, day, hours, minutes, seconds, milliseconds);
```

Date Object

- Var d=new date()
 - getDate() → Returns the day of the month (from 1-31)
 - getDay() → Returns the day of the week (from 0-6)
 - getFullYear() → Returns the year (four digits)
 - getHours() → Returns the hour (from 0-23)
 - getMinutes() → Returns the minutes (from 0-59)
 - getMonth() → Returns the month (from 0-11)
 - getSeconds() → Returns the seconds (from 0-59)
 - getTime() → Returns the number of milliseconds since midnight Jan 1, 1970

Digital clock

```
<html>
<head>
<script>
function startTime() {
    var today=new Date();
    var h=today.getHours();
    var m=today.getMinutes();
    var s=today.getSeconds();
    m = checkTime(m);
    s = checkTime(s);
    document.getElementById('txt').innerHTML = h+":"+m+": "+s;
    var t = setTimeout(function(){startTime()},500);
}
function checkTime(i) {
    if (i<10) {i = "0" + i}; // add zero in front of numbers < 10
    return i;
}
</script>
</head>
<body onload="startTime()">
<div id="txt"></div>
</body>
</html>
```

2.String object

- A JavaScript string stores a series of characters.
- A string can be any text inside double or single quotes.
- String indexes are zero-based:
 - The first character is in position 0, the second in 1, and so on.

JavaScript String length Property

Example

Return the number of characters in a string:

```
var str = "Hello World!";  
var n = str.length;
```

The result of *n* will be:

12

Method	Description
<u>charAt()</u>	Returns the character at the specified index (position)
<u>charCodeAt()</u>	Returns the Unicode of the character at the specified index
<u>concat()</u>	Joins two or more strings, and returns a new joined strings
<u>endsWith()</u>	Checks whether a string ends with specified string/characters
<u>fromCharCode()</u>	Converts Unicode values to characters
<u>includes()</u>	Checks whether a string contains the specified string/characters
<u>indexOf()</u>	Returns the position of the first found occurrence of a specified value in a string
<u>lastIndexOf()</u>	Returns the position of the last found occurrence of a specified value in a string
<u>localeCompare()</u>	Compares two strings in the current locale
<u>match()</u>	Searches a string for a match against a regular expression, and returns the matches
<u>repeat()</u>	Returns a new string with a specified number of copies of an existing string
<u>replace()</u>	Searches a string for a specified value, or a regular expression, and returns a new string where the specified values are replaced

<u>search()</u>	Searches a string for a specified value, or regular expression, and returns the position of the match
<u>slice()</u>	Extracts a part of a string and returns a new string
<u>split()</u>	Splits a string into an array of substrings
<u>startsWith()</u>	Checks whether a string begins with specified characters
<u>substr()</u>	Extracts the characters from a string, beginning at a specified start position, and through the specified number of character
<u>substring()</u>	Extracts the characters from a string, between two specified indices
<u>toLocaleLowerCase()</u>	Converts a string to lowercase letters, according to the host's locale
<u>toLocaleUpperCase()</u>	Converts a string to uppercase letters, according to the host's locale
<u>toLowerCase()</u>	Converts a string to lowercase letters
<u>toString()</u>	Returns the value of a String object
<u>toUpperCase()</u>	Converts a string to uppercase letters
<u>trim()</u>	Removes whitespace from both ends of a string
<u>valueOf()</u>	Returns the primitive value of a String object

Comparing strings

C developers have the `strcmp()` function for comparing strings. In JavaScript, you just use the less-than and greater-than operators:

```
1  var a = 'a';
2  var b = 'b';
3  if (a < b) { // true
4      console.log(a + ' is less than ' + b);
5  } else if (a > b) {
6      console.log(a + ' is greater than ' + b);
7  } else {
8      console.log(a + ' and ' + b + ' are equal.');
```

The `split()` method splits a `String` object into an array of strings by separating the string into substrings, using a specified separator string to determine where to make each split.



JavaScript Demo: String.split()

```
1 var str = 'The quick brown fox jumps over the lazy dog.';
2
3 var words = str.split(' ');
4 console.log(words[3]);
5 // expected output: "fox"
6
7 var chars = str.split('');
8 console.log(chars[8]);
9 // expected output: "k"
10
11 var strCopy = str.split();
12 console.log(strCopy);
13 // expected output: Array ["The quick brown fox jumps over the lazy dog."]
```

Run >

Reset

> "fox"

> "k"

> Array ["The quick brown fox jumps over the lazy dog."]

⚠ Warning: Although `String.prototype.substr(...)` is not strictly deprecated (as in "removed from the Web standards"), it is considered a **legacy function** and should be avoided when possible. It is not part of the core JavaScript language and may be removed in the future. If at all possible, use the `substring()` method instead.

The `substr()` method returns a portion of the string, starting at the specified index and extending for a given number of characters afterward.



JavaScript Demo: String.substr()

```
1 var str = 'Mozilla';
2
3 console.log(str.substr(1, 2));
4 // expected output: "oz"
5
6 console.log(str.substr(2));
7 // expected output: "zilla"
```

The `substring()` method returns the part of the `string` between the start and end indexes, or to the end of the string.



JavaScript Demo: String.substring()

```
1 var str = 'Mozilla';  
2  
3 console.log(str.substring(1, 3));  
4 // expected output: "oz"  
5  
6 console.log(str.substring(2));  
7 // expected output: "zilla"
```

Using `endsWith()`

```
1 var str = 'To be, or not to be, that is the question.';
2
3 console.log(str.endsWith('question.')); // true
4 console.log(str.endsWith('to be'));    // false
5 console.log(str.endsWith('to be', 19)); // true
```


3.Array object

- An object's job is that of storing and allowing for the easy manipulation of data.
- We can safely say that the **Array object** performs both tasks wonderfully.

How do you create an Array object?

- use *new Array()*

```
var colors = new Array();  
colors[0] = "green";  
colors[1] = "red";
```

- use *new Array(item0, item1, item2 ...)*

```
var colors = new Array("green", "red");
```

- use the *literal array* approach

```
var colors = ["green", "red"];
```

How to access data inside the Array object

```
var colors = ["green", "red", "yellow", "orange"];  
var red = colors[1];  
document.write(red);
```

How to access all elements from the array

```
var colors = ["green", "red", "yellow", "orange"];
```

```
for (var i=0; i < colors.length; i++)  
{  
document.write(colors[i] + "<br />");  
}
```

Another method

```
var colors = ["green", "red", "yellow", "orange"];  
for (c in colors)  
  { //c is the variable that marks the index position  
    document.write(colors[c] + "<br />");  
  }
```

Sorting the array elements

```
var colors = ["green", "red", "yellow", "orange",];
```

```
var ascendingColors = colors.sort();
```

```
//display each value
```

```
document.write(ascendingColors);
```

```
var sum = 0;           // The sum starts off at 0  
count = 0;  
do  
{  
if (listOfNumbers[count] % 5 == 0)  
sum += listOfNumbers[count]; // Add the value to the sum  
count++;  
}while (count < 83);
```

Array Object Methods

Method	Description
<u>concat()</u>	Joins two or more arrays, and returns a copy of the joined arrays
<u>indexOf()</u>	Search the array for an element and returns its position
<u>join()</u>	Joins all elements of an array into a string
<u>lastIndexOf()</u>	Search the array for an element, starting at the end, and returns its position
<u>pop()</u>	Removes the last element of an array, and returns that element
<u>push()</u>	Adds new elements to the end of an array, and returns the new length
<u>reverse()</u>	Reverses the order of the elements in an array
<u>shift()</u>	Removes the first element of an array, and returns that element
<u>slice()</u>	Selects a part of an array, and returns the new array
<u>sort()</u>	Sorts the elements of an array
<u>splice()</u>	Adds/Removes elements from an array
<u>toString()</u>	Converts an array to a string, and returns the result
<u>unshift()</u>	Adds new elements to the beginning of an array, and returns the new length
<u>valueOf()</u>	Returns the primitive value of an array

Concat()



JavaScript Demo: Array.concat()

```
1 var array1 = ['a', 'b', 'c'];
2 var array2 = ['d', 'e', 'f'];
3
4 console.log(array1.concat(array2));
5 // expected output: Array ["a", "b", "c", "d", "e", "f"]
```

Concat()

Concatenating three arrays

The following code concatenates three arrays:

```
1  const num1 = [1, 2, 3];  
2  const num2 = [4, 5, 6];  
3  const num3 = [7, 8, 9];  
4  
5  const numbers = num1.concat(num2, num3);  
6  
7  console.log(numbers);  
8  // results in [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Fill()

The `fill()` method fills all the elements of an array from a start index to an end index with a static value. The end index is not included.



JavaScript Demo: Array.fill()

```
1 var array1 = [1, 2, 3, 4];
2
3 // fill with 0 from position 2 until position 4
4 console.log(array1.fill(0, 2, 4));
5 // expected output: [1, 2, 0, 0]
6
7 // fill with 5 from position 1
8 console.log(array1.fill(5, 1));
9 // expected output: [1, 5, 5, 5]
10
11 console.log(array1.fill(6));
12 // expected output: [6, 6, 6, 6]
```

Filter()

The `filter()` method creates a new array with all elements that pass the test implemented by the provided function.



JavaScript Demo: Array.filter()

```
1 var words = ['spray', 'limit', 'elite', 'exuberant', 'destruction', 'present'];
2
3 const result = words.filter(word => word.length > 6);
4
5 console.log(result);
6 // expected output: Array ["exuberant", "destruction", "present"]
-
```

Filter()

Filtering out all small values [↗](#)

The following example uses `filter()` to create a filtered array that has all elements with values less than 10 removed.

```
1 function isBigEnough(value) {  
2     return value >= 10;  
3 }  
4  
5 var filtered = [12, 5, 8, 130, 44].filter(isBigEnough);  
6 // filtered is [12, 130, 44]
```

Map()

The `map()` method creates a new array with the results of calling a provided function on every element in the calling array.



JavaScript Demo: Array.map()

```
1 var array1 = [1, 4, 9, 16];  
2  
3 // pass a function to map  
4 const map1 = array1.map(x => x * 2);  
5  
6 console.log(map1);  
7 // expected output: Array [2, 8, 18, 32]
```

Map()

Mapping an array of numbers to an array of square roots

The following code takes an array of numbers and creates a new array containing the square roots of the numbers in the first array.

```
1 | var numbers = [1, 4, 9];  
2 | var roots = numbers.map(Math.sqrt);  
3 | // roots is now [1, 2, 3]  
4 | // numbers is still [1, 4, 9]
```

slice

```
programs > javascript > JS slicepgm.js > ...  
1   const animals = ['ant', 'bison', 'camel', 'duck', 'elephant'];  
2  
3   console.log(animals.slice(2));  
4  
5  
6   var slicer=animals.slice(2, 4);  
7   console.log(slicer);  
8  
9  
10  console.log(animals.slice(1, 5));  
11  console.log(animals);
```

```
PS D:\mekala_f\Winter Semester20-21\CSE3002\Programs\programs\javascript> node slicepgm.  
[ 'camel', 'duck', 'elephant' ]  
[ 'camel', 'duck' ]  
[ 'bison', 'camel', 'duck', 'elephant' ]  
[ 'ant', 'bison', 'camel', 'duck', 'elephant' ]
```


splice

- The splice() method changes the contents of an array by removing or replacing existing elements and/or adding new element

```
var months = ['Jan', 'March', 'April', 'June'];
months.splice(1, 0, 'Feb');
// inserts at index 1
console.log(months);
// expected output: Array ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, 'May');
// replaces 1 element at index 4
console.log(months);
// expected output: Array ["Jan", "Feb", "March", "April", "May"]
```

Example

At position 2, add the new items, and remove 1 item:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.splice(2,1,"Lemon","Kiwi");
```

The result of *fruits* will be:

Banana, Orange, Lemon, Kiwi, Mango

Example

At position 2, remove 2 items:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.splice(2,2);
```

The result of *fruits* will be:

Banana, Orange

pop

Example

Remove the last element of an array:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.pop();
```

The result of *fruits* will be:

```
Banana, Orange, Apple
```

shift

Example

Remove the first item of an array:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.shift()
```

The result of *fruits* will be:

```
Orange,Apple,Mango
```

push

Example

Add a new item to an array:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.push("Kiwi")
```

The result of *fruits* will be:

```
Banana, Orange, Apple, Mango, Kiwi
```

unshift

Example

Add new items to the beginning of an array:

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];  
fruits.unshift("Lemon","Pineapple");
```

The result of *fruits* will be:

Lemon,Pineapple,Banana,Orange,Apple,Mango

Math Properties

Property	Description
<u>E</u>	Returns Euler's number (approx. 2.718)
<u>LN2</u>	Returns the natural logarithm of 2 (approx. 0.693)
<u>LN10</u>	Returns the natural logarithm of 10 (approx. 2.302)
<u>LOG2E</u>	Returns the base-2 logarithm of E (approx. 1.442)
<u>LOG10E</u>	Returns the base-10 logarithm of E (approx. 0.434)
<u>PI</u>	Returns PI (approx. 3.14)
<u>SQRT1_2</u>	Returns the square root of 1/2 (approx. 0.707)
<u>SQRT2</u>	Returns the square root of 2 (approx. 1.414)

Math Object Methods

<u>pow(x,y)</u>	Returns the value of x to the power of y
<u>random()</u>	Returns a random number between 0 and 1
<u>round(x)</u>	Rounds x to the nearest integer
<u>sin(x)</u>	Returns the sine of x (x is in radians)
<u>sqrt(x)</u>	Returns the square root of x
<u>tan(x)</u>	Returns the tangent of an angle

Method	Description
<u>abs(x)</u>	Returns the absolute value of x
<u>acos(x)</u>	Returns the arccosine of x, in radians
<u>asin(x)</u>	Returns the arcsine of x, in radians
<u>atan(x)</u>	Returns the arctangent of x as a numeric value between -PI/2 and PI/2 radians
<u>atan2(y,x)</u>	Returns the arctangent of the quotient of its arguments
<u>ceil(x)</u>	Returns x, rounded upwards to the nearest integer
<u>cos(x)</u>	Returns the cosine of x (x is in radians)
<u>exp(x)</u>	Returns the value of E^x
<u>floor(x)</u>	Returns x, rounded downwards to the nearest integer
<u>log(x)</u>	Returns the natural logarithm (base E) of x
<u>max(x,y,z,...,n)</u>	Returns the number with the highest value
<u>min(x,y,z,...,n)</u>	Returns the number with the lowest value
<u>pow(x,y)</u>	Returns the value of x to the power of y

JavaScript Numbers

JavaScript has only one type of number.

Numbers can be written with, or without, decimals:

Example

```
var x = 3.14;    // A number with decimals  
var y = 34;      // A number without decimals
```

Extra large or extra small numbers can be written with scientific (exponent) notation:

Example

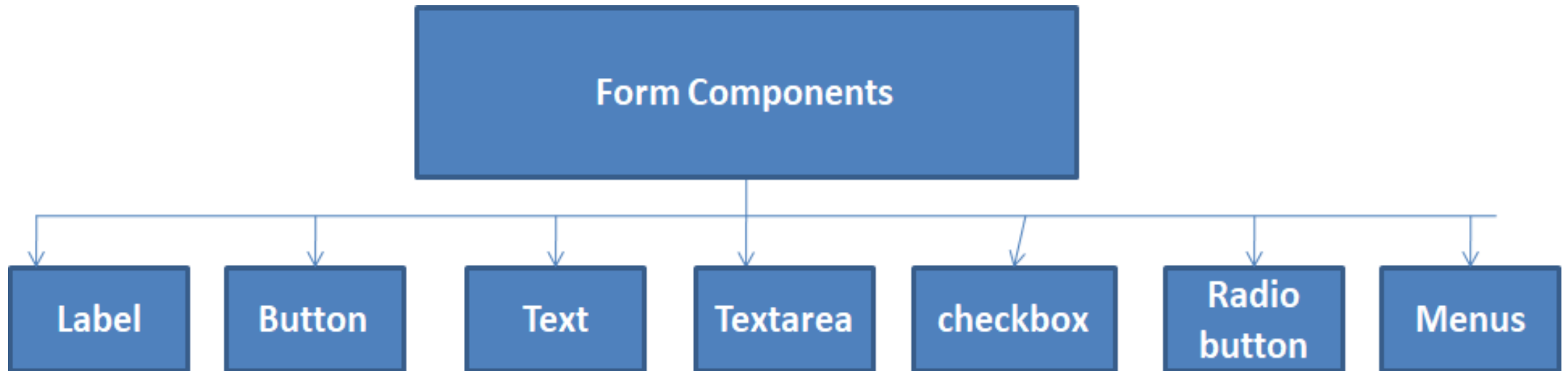
```
var x = 123e5;    // 12300000  
var y = 123e-5;   // 0.00123
```

Number Methods

Method	Description
<u>isFinite()</u>	Checks whether a value is a finite number
<u>isInteger()</u>	Checks whether a value is an integer
<u>isNaN()</u>	Checks whether a value is Number.NaN
<u>isSafeInteger()</u>	Checks whether a value is a safe integer
<u>toExponential(x)</u>	Converts a number into an exponential notation
<u>toFixed(x)</u>	Formats a number with x numbers of digits after the decimal point
<u>toPrecision(x)</u>	Formats a number to x length
<u>toString()</u>	Converts a number to a string
<u>valueOf()</u>	Returns the primitive value of a number

Forms Creation

- HTML5 provides a mechanism, **called a form**, for collecting data from a user.
- Data that users enter on a web page is normally sent to a web server that provides access to a site's resources



```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <meta charset = "utf-8">
```

```
    <title>form</title>
```

```
</head>
```

```
<body>
```

```
<h1>Feedback Form</h1><br>
```

```
<p>Please fill out this form to help us improve our site<br><br><br><form>
```

```
Name:<input type="text" name="firstname" value=""><br><br> —————> Text box
```

```
Comments:<br><textarea cols="40" rows="5" name="address"></textarea><br><br> —————> Text area
```

```
E-Mail address:<input type="text" name="email" value=""><br><br> —————> Text box
```

```
Things you liked:<br>
```

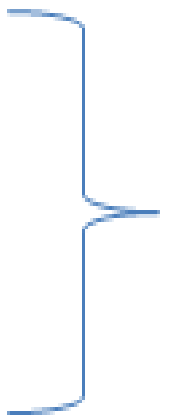
```
<input type="checkbox" name="checkbox1" value="site design" checked="checked">site design
```

```
<input type="checkbox" name="checkbox2" value="links">links
```

```
<input type="checkbox" name="checkbox3" value="ease of use">ease of use Check box
```

```
<input type="checkbox" name="checkbox4" value="images">Images
```

```
<input type="checkbox" name="checkbox5" value="source code">Source code<br><br>
```



How did you get to our site:


```
<input type="radio" name="radio1" value="search engine" checked="checked"> search engine  
<input type="radio" name="radio2" value="Links from another site">Links from another site  
<input type="radio" name="radio3" value="Deitel.com website">Deitel.com website  
<input type="radio" name="radio4" value="Reference in a book">Reference in a book  
<input type="radio" name="radio5" value="other">other<br><br>
```

Radio button

Rate our website:

```
<select>  
<option selected>Amazing</option>  
<option>5</option>  
<option>4</option>  
<option>3</option>  
<option>2</option>  
<option>1</option>  
</select><br><br>  
<input type="submit" value="submit">  
<input type="reset" value="clear">  
</forms></body></html>
```

Menus

Button



Feedback Form

Please fill out this form to help us improve our site

Name:

Comments:

E-Mail address:

Things you liked:

☒ site design ☐ links ☐ ease of use ☐ Images ☐ Source code

How did you get to our site:

☒ search engine ☐ Links from another site ☐ Deitel.com website ☐ Reference in a book ☐ other

Rate our website:

submit

clear

Hidden field

- The `<input type="hidden">` defines a hidden input field.
- A hidden field let web developers include data that cannot be seen or modified by users when a form is submitted.
- A hidden field often stores what database record that needs to be updated when the form is submitted.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset = "utf-8">
    <title>form</title>
</head>
<body>
    <h1>A Hidden Field</h1>
    <form method = "post" action = "http://www.deitel.com">
        <label>Name:</label>
        <input type="text" name="fname" value=""><br><br>
        <label>Password</label>
        <input type="password" name="p1" value="">
        <input type="hidden" name = "recipient" value = "deitel@deitel.com"><br><br>
        <input type="submit" value="Submit">
        <input type="reset" value="clear">
    </form>
</body>
```

A Hidden Field

Name:

Password:



[Home](#) [Contact Us](#) [Blog](#) [Videos](#) [Training](#) [Books](#) [FAQ](#) [About](#) [Search](#)

[Click to View Live Trainings and Schedules](#)



Our New Java Books and Videos

Java How to Program: An Objects-Natural Approach,

HTML5 Form elements

```
<!DOCTYPE html>
<html>
<body>
<form>
  select color:<input type = "color"><br>
  select date:<input type = "date"><br>
  enter email:<input type = "email" placeholder = "name@domain.com" required><br>
  select month:<input type = "month"/> <br>
  select number:<input type = "number" min = "0" max = "7" step = "1" value="0"/> <br>
  select range:<input type = "range" min = "0" max = "20" value = "10"/><br>
  search:<input type = "search" placeholder = "search query" /><br>
  select time:<input type = "time" /> <br>
  select week: <input type = "week" /><br>
</form>
</body>
</html>
```

select color:

select date: mm / dd / yyyy



enter email: name@domain.com

select month: -----



select number: 0

select range:

search: search query

select time: -- : -- --



select week: Week -- , -----



select

FORMS

- An HTML form is used to collect user input.
- The input from the user is generally sent to a server (Web servers, Mail clients, etc).
- HTML forms, defined using the <form> tag
- They include interactive controls like text fields, emails, passwords, checkboxes, radios, and buttons.

<form>

form elements, such as text box, textarea, etc.

</form>

```
<form action="submit.php" method="post">  
</form>
```

action → Where the form data is sent.

Method → How the data is sent (GET/ POST)

Use POST for passwords or sensitive data.

The image shows a visual representation of an HTML form. It contains an 'Email' field with the text 'example@gmail.com', a 'Password' field with the placeholder text 'Enter Your Password' and a toggle icon, a 'Remember Me' checkbox, a 'Forgot Password?' link, and a blue 'Sign In' button.

<input> Element

- Used inside a <form> to take user input.

```
<input type="text">
```

<label> Element

- Provides a caption for form inputs

```
label>Name:</label>  
<input type="text">
```

Name:

- There are 2 ways to use <label>

1) Using for Attribute (with id)

```
<label for="username">Enter Name:</label>  
<input type="text" id="username">
```

2) Wrapping Input Inside <label>

```
<label>  
  Enter Name:  
  <input type="text">  
</label>
```

```
<input type="text"/>
```

Some text

```
<input type="password"/>
```

.....

```
<input type="button"/>
```

Button

```
<input type="checkbox"/>
```

☐

User

☒

Admin

```
<input type="color"/>
```

```
<input type="date"/>
```

02/15/2023



```
<input type="time"/>
```

10:21 AM



```
<input type="email" />
```

xyz@sample.com

```
<input type="file"/>
```

Choose File

No file chosen

```
<input type="url"/>
```

https://www.siteurl.com

```
<input type="week"/>
```

Week 37, 2024



```
<input type="number"/>
```

4

```
<input type="month"/>
```

July 2024



```
<input type="radio"/>
```

☐

HTML

☒

CSS

```
<input type="range"/>
```

```
<input type="reset"/>
```

Reset

```
<input type="search"/>
```

Search

```
<input type="submit"/>
```

Submit


```
<html> <body>
  <h2>Addition</h2>

  <label>Enter First Number:</label>
  <input type="number" id="num1">
  <br><br>

  <label>Enter Second Number:</label>
  <input type="number" id="num2">
  <br><br>

  <label>Sum:</label>
  <input type="text" id="result">
  <br><br>

  <button onclick="addNumbers()">Add</button>
  <br><br>
<script>
  function addNumbers() {
    let number1 = document.getElementById("num1").value;
    let number2 = document.getElementById("num2").value;
    let sum = Number(number1) + Number(number2);
    document.getElementById("result").value = sum;
  }
</script> </body> </html>
```


Text Control

- A **text control** is an input field that allows users to enter text.

`<input type="text">`

Attribute	Purpose	Simple Example
type	Defines input type	<code><input type="text"></code>
name	Sends data to server	<code><input type="text" name="username"></code>
id	Connects with label / JS	<code><input type="text" id="user"></code>
placeholder	Shows hint text	<code><input type="text" placeholder="Enter name"></code>
value	Default text inside input	<code><input type="text" value="John"></code>
maxlength	Limits characters	<code><input type="text" maxlength="10"></code>
size	Controls width	<code><input type="text" size="30"></code>
required	Makes field compulsory	<code><input type="text" required></code>
readonly	Cannot edit value	<code><input type="text" value="Admin" readonly></code>
disabled	Completely disabled	<code><input type="text" disabled></code>

```

<html>
<body>
<h2>Text Control Attributes Demo</h2>
<form>
  <label for="username">Username:</label>
<input type="text" id="username" name="username">
  <br><br>

  <label>Placeholder:</label>
<input type="text" placeholder="Enter your name">
  <br><br>

<label>Default Value:</label>
  <input type="text" value="John">
  <br><br>

<label>Maxlength (5 characters only):</label>
  <input type="text" maxlength="5">
  <br><br>

<label>Size (Width 30):</label>
  <input type="text" size="30">
  <br><br>

<label>Required Field:</label>
  <input type="text" required>
  <br><br>

```

```

<label>Readonly Field:</label>
<input type="text" value="Admin" readonly>
<br><br>

<label>Disabled Field:</label>
  <input type="text" value="Disabled"
disabled>
  <br><br>

  <input type="submit" value="Submit">

</form>
</body>
</html>

```

Text Control Attributes

Username:

Placeholder:

Default Value:

Maxlength (5 characters only):

Size (Width 30):

Required Field:

Readonly Field:

Disabled Field:

Accessing a Form's Control Values

- When a user types into a form (like `<input>`, `<textarea>`, `<select>`), you can access those values using JavaScript.

1. `document.getElementById()`

- Used to select (or access) an HTML element using its **id attribute**.
- It is widely used for:
 - Changing content
 - Modifying styles
 - Handling events
 - Creating interactive web pages

```
document.getElementById("idName");
```

Part	Description
document	Represents the entire HTML document
getElementById()	Method used to find an element
"idName"	The id value of the HTML element (written as a string)

```
<html> <body>
  <p id="demo">This is a paragraph.</p>
  <button onclick="changeColor()">Change Color</button>
  <script>
    function changeColor() {
      document.getElementById("demo").style.color = "red";
    }
  </script> </body> </html>
```

2. document.forms

- A collection (array-like object) of all `<form>` elements in the HTML document.
- Each form can be accessed:
 - *By name*
 - *By index number*

document.forms["formName"]

document.forms[0]

document.forms["formName"]["fieldName"].value

Login Form

Email:

```
<html> <body>
```

```
  <h2>Login Form</h2>
```

```
  <form name="loginForm">
```

```
    Email: <input type="text" name="email"  
           placeholder="Enter your email">
```

```
    <button type="button" onclick="getEmail()">  
      Submit</button>
```

```
  </form>
```

```
  <script>
```

```
    function getEmail() {
```

```
      var emailValue =
```

```
        document.forms["loginForm"]["email"].value;
```

```
      if (emailValue === "") {
```

```
        alert("Please enter your email.");
```

```
      } else {
```

```
        alert("Email: " + emailValue);
```

```
      } }
```

```
    </script> </body> </html>
```

3. reset() Method

- Resets all form fields to their default values.
- Default values are the ones written in the HTML when the page loads

```
<html>
<body>
  <h2>Reset Form</h2>
  <form id="myForm">
    Name: <input type="text" value="Baiju"><br><br>
    Email: <input type="text" ><br><br>
    <button type="button" onclick="resetForm()">Reset</button>
  </form>
  <script>
    function resetForm() {
      document.getElementById("myForm").reset();
    }
  </script> </body> </html>
```

3. focus()

- Places the cursor inside an input field.

```
<html>
<body>

  Username: <input type="text" id="un">
  <script>
    document.getElementById("un").focus();
  </script>
</body>
</html>
```

Username:

```
<html>
<body>

  Username: <input type="text" id="un">
  <script>
    document.getElementById("un").value;
  </script>
</body>
</html>
```

Username:

Form Object

- The Form object represents an HTML <form> element.

Access a Form Object

- You can access a <form> element by using getElementById():

```
var x = document.getElementById("myForm");
```

Accessing Form elements

```
<html>
<head>
  <title>Find In JavaScript</title>
</head>
<body>
  <form>
    Enter your name:
    <input type="text" name="yourname"> <br> <br>
    Enter Password:
    <input type="password" name="yourpassword"> <br> <br>
    <input type="button" onclick="value_show()">
  </form>
  <script type="text/javascript">
function value_show()
{
  var number_elements=document.forms[0].length;
  var element_name=document.forms[0].elements[0].name;
  var element_value=document.forms[0].elements[0].value;
  document.write(number_elements);
  document.write(element_name);
}
```


Accessing all Form elements

```
<html>

<head>

  <title>Find In JavaScript</title>

</head>

<body>

  <form id="frm1" name="frm1">

    Enter your name:

    <input type="text" name="yourname" value="gsjg"> <br> <br>

    Enter Password:

    <input type="password" name="yourpassword" value="khsh"> <br> <br>

    <input type="submit" name="submit" value="Register">

  </form>

  <input type="button" onclick="newfun()">

  function newfun()

  {

    var x = document.getElementById("frm1");

    var text = "";

    var i;

    for (i = 0; i < x.length ;i++) {

      text += x.elements[i].value + "<br>";

    }

  }
```

Form Object Properties

- **acceptCharset** → Sets or returns the value of the accept-charset attribute in a form.
Eg: Character encoding for Unicode (UTF-8) , Latin alphabet (ISO-8859-1)
- **action** → Sets or returns the value of the action attribute in a form
- **enctype** → Sets or returns the value of the enctype attribute in a form
- **length** → Returns the number of elements in a form
- **method** → Sets or returns the value of the method attribute in a form (get or post)
- **name** → Sets or returns the value of the name attribute in a form
- **target** → Sets or returns the value of the target attribute in a form (_blank, _top, _parent, _self, framename)

Form Object Methods

1. **reset()** → Resets a form

Eg : `document.getElementById("form1").reset();`

1. **submit()** → Submits a form

Form Object Events

1. **onreset** → The reset button is clicked
2. **onsubmit** → The submit button is clicked

output

First name: Mahatma

Last name: Gandhiji

Submit

fname:Mahatma

lname:Gandhiji

Submit:Submit

Example

```
<html> <body>
<form id="form1" action="">
First name: <input type="text" name="fname" value="Mahatma">
<br>
Last name: <input type="text" name="lname" value="Gandhiji"><br>
<input type="button" value="Submit" name="Submit">
</form>

<script type="text/javascript">
var x=document.getElementById("form1");
for (var i=0;i<x.length;i++)
{
document.write(x.elements[i].name+": "+x.elements[i].value + "<br>");
}
</script> </body> </html>
```

Reset()

```
<html> <head>
<script type="text/javascript">
function formReset()
{ document.getElementById("frm1").reset(); }
</script> </head> <body>
<p>Enter some text in the fields below, then press the "Reset
  form" button to reset the form.</p>
<form id="frm1">
First name: <input type="text" name="fname" ><br >
Last name: <input type="text" name="lname" ><br ><br >
<input type="button" onclick="formReset()" value="Reset form" >
</form> </body> </html>
```

Submit()

```
<html>
```

```
<body>
```

```
<p>Enter some text in the fields below, then press the "Submit form" button to  
submit the form.</p>
```

```
<form id="myForm" action="form_action.asp">
```

```
  First name: <input type="text" name="fname"><br>
```

```
  Last name: <input type="text" name="lname"><br><br>
```

```
  <input type="button" onclick="myFunction()" value="Submit form">
```

```
</form>
```

```
<script>
```

```
function myFunction() {
```

```
  document.getElementById("myForm").submit();
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Form validation using Javascript

Accessing form Elements –Text Box

```
<form name="test" onsubmit="return  
checkinput()">
```

```
Username:<input type="text" name="username"><br>
```

```
Password:<input type="password" maxlength="6"  
name="upass"><br>
```

```
Confirm Password:<input type="password"  
name="cpass">
```

```
<input type="submit" value="check">
```

```
</form>
```

Accessing form Elements-Text Box

```
<script>
function checkinput()
{

if (document.test.username.value=="")
{
    alert("Please enter the User
Name") ;
    return false;
}

}
</script>
```

Accessing form Elements Text Box

```
<script>
function checkinput()
{
    if(document.test.username.value=="")
    {
        alert("Please enter the User Name");
        return false;
    }
    if(document.test.upass.value==document.test.cpass.value)
    {

        alert("Thank you for Registration");
        return true;
    }
    else
    {
        alert("passwords dont match");
        return false;
    }
}
```

Accessing form elements using ID

```
<form name="test" onsubmit="return checkinput()">
```

```
Username:<input type="text" name="username"  
id="123"><br>
```

```
Password:<input type="password" maxlength="6"  
name="upass" id="456"><br>
```

```
<input type="submit" value="check">  
</form>
```

Accessing form elements using ID

```
function checkinput()
{
var username = document.getElementById("123").value;
var password = document.getElementById("456").value;
if (username=="")
{
    alert("Please enter a username");
    return false;
}
if (password=="")
{
    alert("Please enter a password");
    return false;
}

    alert("Thanks for registration");
    return true;
}
```

Accessing the Drop Down /Select Box

```
</script>
<form name="test" onsubmit="return checkinput()">
<select name = "dropbox">
<option value = "">Select an answer </option>
<option value="RED">RED</option>
<option value = "BLUE">BLUE</option>
<option value = "GREEN">GREEN</option>
</select>
<input type="submit" value="Submit">
</form>
```

```
function checkinput()
{
    if(document.test.dropbox.selectedIndex == 0)
    {

        alert("Please select a value" );

        return false;

    }
    else
    {
alert("Thanks for the Selection " +
document.test.dropbox.options[test.dropbox.selectedIndex] .
value) ;

        return true;

    }
}
```

Accessing Radio options using id

```
<form name="test" onsubmit="return checkinput()">
```

Gender:

```
</br>
```

```
<input type = "radio" value ="Male" name="gender" id="male"/>Male
```

```
</br>
```

```
<input type = "radio" value = "female" name = "gender"  
id="female"/>Female</br>
```

```
</br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```



```
<script>
function checkinput()
{
    var check1=document.getElementById("male").checked;
    var check2=document.getElementById("female").checked;
    if((check1==false) && (check2==false))
    {
        alert("please fill in some option");
        return false;
    }
    else
    {
        alert("thanks for your selection");
        return true;
    }
}
</script>
```

Accessing radio option value

```
<script>
function checkinput()
{
var i;
for(i=0;i<document.test.gender.length;i++)
{
    if(document.test.gender[i].checked)
        {
            alert(document.test.gender[i].value);
        }
}
}
</script>
```

Accessing Checkboxes

```
<form name="test" onsubmit="return checkinput()">
Courses Completed:<br>
<input type = "checkbox" name="course"
value="SoftwareEngineering" id="ss1"> Software Engineering
</br>
<input type = "checkbox" name = "course"
value="DatabaseManagementSystems"> Database Management
Systems
</br>
<input type = "checkbox" name = "course"
value="ScriptingLanguages"> Scripting Languages
</br></br>
<input type ="submit">
```

```
<script>
function checkinput()
{
    var test1 = document.getElementById("ss1").checked;
    var test2 = document.getElementById("ss2").checked;
    var test3 = document.getElementById("ss3").checked;
    if ((test1==false) && (test2==false) && (test3==false))
    {
        alert("please select a value");
        return false;
    }
    else
    {
        alert("thanks for your selection");
        return true;
    }
}
</script></
```

Accessing Checkbox Value

```
<script>
function checkinput()
{
    for(i=0;i<document.test.course.length;i++)
    {
        if(document.test.course[i].checked)
        {
            alert(document.test.course[i].value);
        }
    }
}
</script>
```

Registration Form

User Id: Required and must be of length 5 to 12.

Password: Required and must be of length 7 to 12.

Name: Required and alphabates only.

Address: Optional.

Country: Required. Must select a country.

ZIP Code: Required. Must be numeric only.

Email: Required. Must be a valid email.

Sex: ☐ Male ☐ Female Required.

Language: ☒ English ☐ Non English Required.

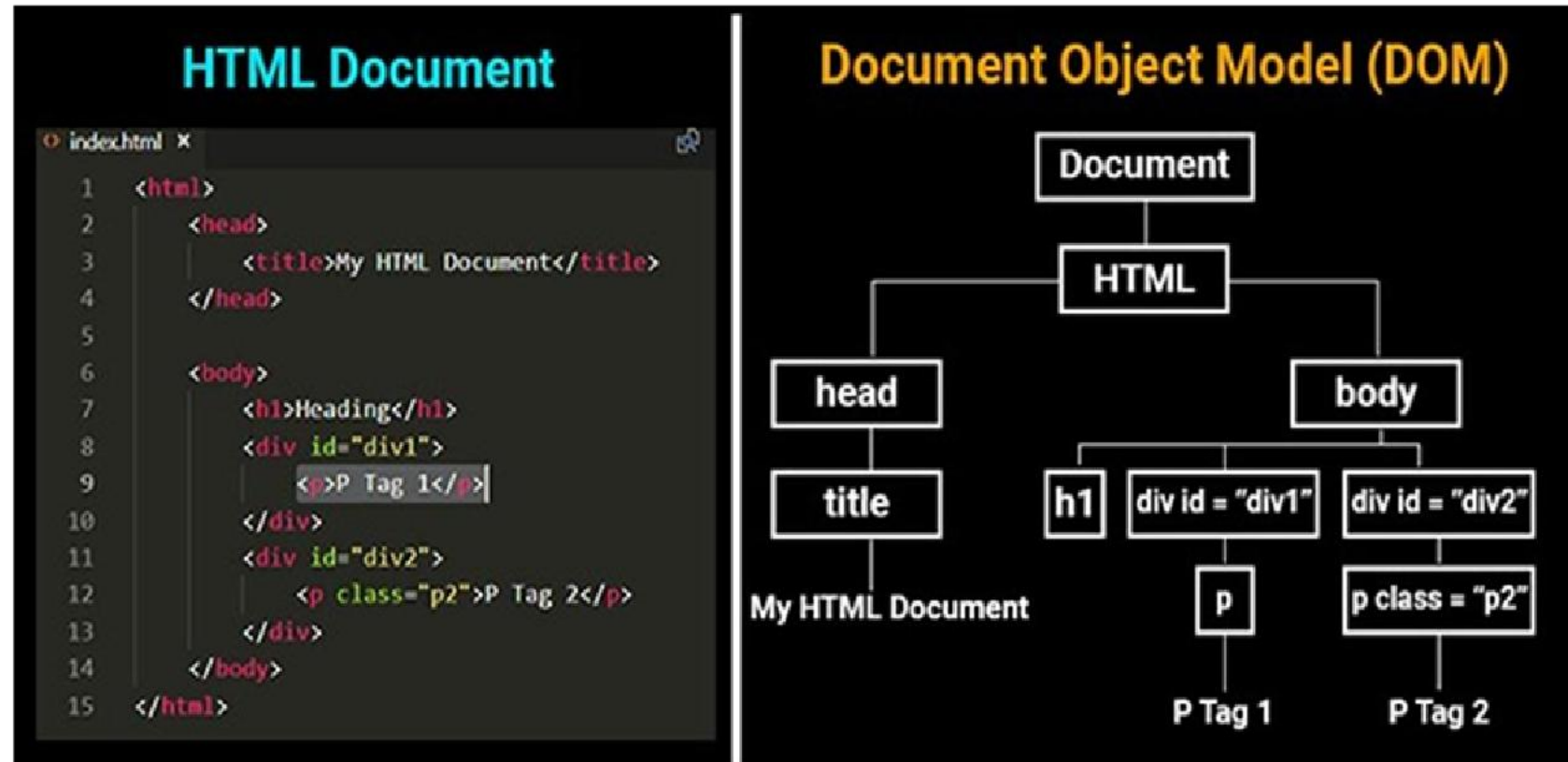
About:

Optional.

Submit

Document Object Model

- The Document Object Model (DOM) is a programming interface that lets JavaScript interact with HTML and XML documents.
- When your browser loads a webpage, it converts the HTML into a tree structure.
- That tree is the DOM.



HTML → Structure

DOM → Live, interactive version of that structure

JavaScript → Controller that manipulates it

Properties of DOM

- DOM properties allow JavaScript to **read and modify** webpage elements dynamically

1. document Object Properties

- The document object represents the entire webpage.

(i) document.title

- Gets or sets the page title

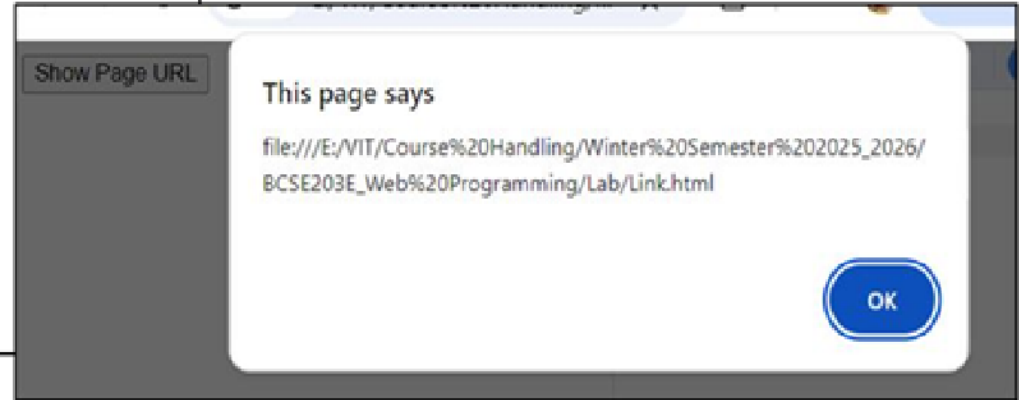
```
<html> <head>
  <title>Faculty Name</title>
</head>
<body>
  <p>Current title: <span id="currentTitle"></span></p>
  <button onclick="changeTitle()">Change Title</button>
<script>
  document.getElementById("currentTitle").innerText = document.title;
  function changeTitle() {
  document.title = "ALICE"; ...
  document.getElementById("currentTitle").innerText = document.title;
  }
  </script> </body> </html>
```


document.URL

- Returns the **full URL (web address)** of the current page.

```
<button onclick="showURL()">Show Page URL</button>
```

```
<script>  
function showURL() {  
    alert(document.URL);  
}  
</script>
```



document.body

document.head

document.images

Access DOM Elements

(i) `getElementById()`

- Used when an element has an **id**.

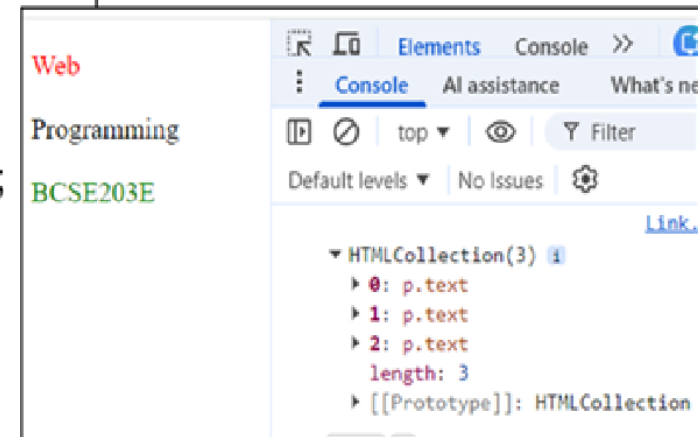
```
<h1 id="title">Hello</h1>
let heading = document.getElementById("title");
console.log(heading);
```

(ii) `getElementsByClassName()`

- Used when elements share a **class name**.

```
<p class="text">Web</p>
<p class="text">Programming</p>
<p class="text">BCSE203E</p>

<script>
let elements = document.getElementsByClassName("text");
    elements[0].style.color = "red";
    elements[2].style.color = "green";
console.log(elements);
</script>
```

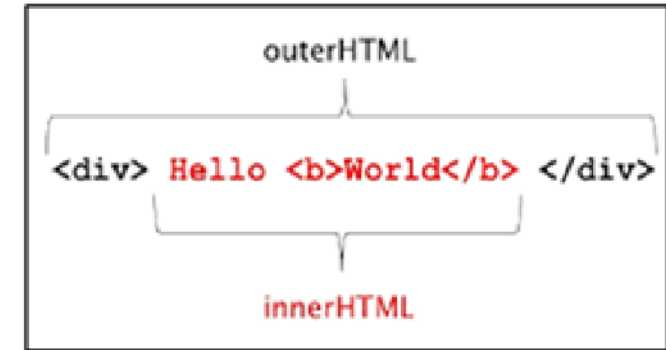


Element Properties

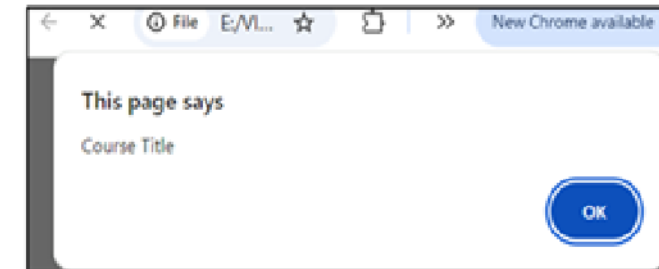
- Each HTML element is represented as an object with many properties.

innerHTML

- Returns or sets HTML content inside an element.



```
<!DOCTYPE html>
<html>
<body>
  <p id="demo">Course Title</p>
<script>
  let element = document.getElementById("demo");
  alert(element.innerHTML);
  element.innerHTML = "<b>Web Program</b>";
</script>
</body> </html>
```



Web Program

The HTML DOM Document Object

- The document object represents your web page.
- If you want to access any element in an HTML page, you always start with accessing the document object.

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

Changing HTML Elements

Property	Description
<i>element.innerHTML = new html content</i>	Change the inner HTML of an element
<i>element.attribute = new value</i>	Change the attribute value of an HTML element
<i>element.style.property = new style</i>	Change the style of an HTML element
Method	Description
<i>element.setAttribute(attribute, value)</i>	Change the attribute value of an HTML element

Adding and Deleting Elements

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

BUTTONS

A button is an **interactive HTML element** that allows the user to **initiate an action** on a web page.

In JavaScript, buttons are used as **event triggers** — they start the execution of JavaScript code when a user interacts with them.

```
<button  
onclick="showMessage()">Click Me</button>
```

```
<script>  
function showMessage()  
{  
  alert("Button was clicked");  
}  
</script>
```

BUTTONS

```
<input type="text" id="name" placeholder="Enter  
your name">
```

```
<button onclick="greet()">Greet</button>
```

```
<script>
```

```
function greet()
```

```
{
```

```
  let user =
```

```
  document.getElementById("name").value;
```

```
  alert("Hello " + user);
```

```
}
```

```
</script>
```


BUTTONS

- A **button** in a web page is an HTML element that allows the user to perform an action when clicked.
- JavaScript is used to make the button **interactive**, meaning it can respond to user actions such as:
 - Clicking
 - Double-clicking
 - Hovering
 - Pressing a key



Creating a Button in HTML

- A button is created using the `<button>` tag

```
<button>Click Me</button>
```



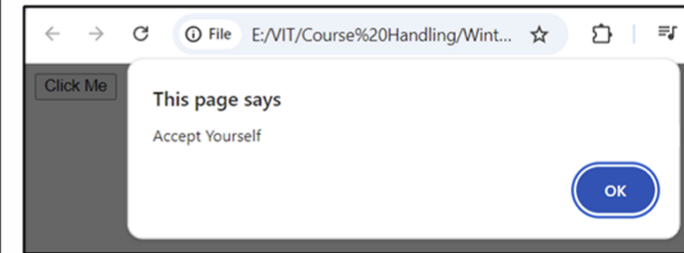
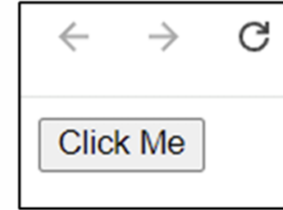
- To make it work, we must use JavaScript.

There are three main ways to use JavaScript with buttons:

1. Using onclick attribute
2. Using a JavaScript function
3. Using addEventListener() (Recommended method)

Method 1: Using onclick Attribute

```
<html> <head>
  <title>Button Example</title>
</head>
<body>
<button onclick="showMessage()">Click Me</button>
<script>
  function showMessage() {
    alert("Accept Yourself");
  }
</script>
</body> </html>
```

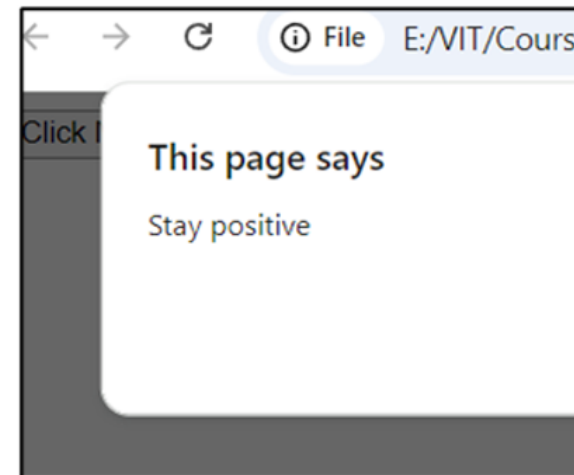


showMessage() → Function that runs when clicked

```
<body>
<button id="myButton">Click Me</button>
<script>
document.getElementById("myButton").onclick = function()
{
    alert("Stay positive");
};
</script>
</body> </html>
```

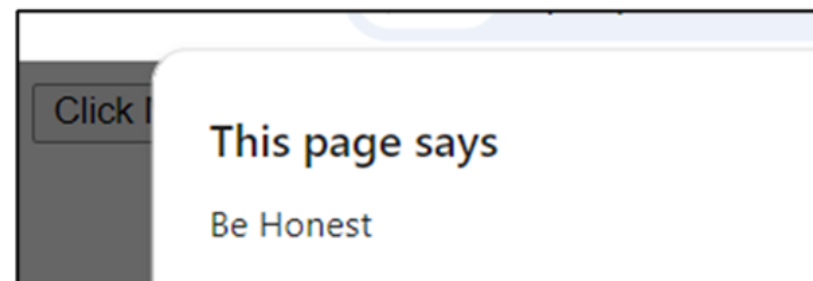
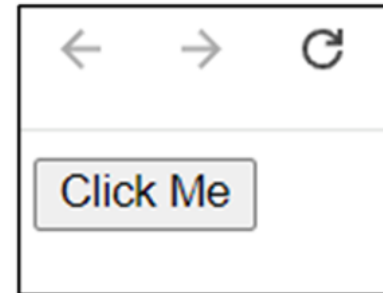
getElementById() → Selects the button

.onclick → Assigns a function to the click event



Method 3: Using addEventListener()

```
<html> <head>
  <title>Button Example</title>
</head>
<body>
<button id="myButton">Click Me</button>
<script>
let button = document.getElementById("myButton");
button.addEventListener("click", function() {
  alert("Be Honest");
});
</script>
</body></html>
```



Accessing a Form's Control Values

- When a user types into a form (like `<input>`, `<textarea>`, `<select>`), you can access those values using JavaScript.

1. `document.getElementById()`

- Used to select (or access) an HTML element using its **id attribute**.
- It is widely used for:
 - Changing content
 - Modifying styles
 - Handling events
 - Creating interactive web pages

```
document.getElementById("idName");
```

Part	Description	<pre><html> <body> <p id="demo">This is a paragraph.</p> <button onclick="changeColor()">Change Color</button> <script> function changeColor() { document.getElementById("demo").style.color = "red"; } </script> </body> </html></pre>
document	Represents the entire HTML document	
getElementById()	Method used to find an element	
"idName"	The id value of the HTML element (written as a string)	

2. document.forms

- A collection (array-like object) of all `<form>` elements in the HTML document.
- Each form can be accessed:
 - *By name*
 - *By index number*

document.forms["formName"]

document.forms[0]

document.forms["formName"]["fieldName"].value

Login Form

Email:

```
<html> <body>
  <h2>Login Form</h2>
  <form name="loginForm">
    Email: <input type="text" name="email"
              placeholder="Enter your email">
    <button type="button" onclick="getEmail()">
      Submit</button>
  </form>
  <script>
    function getEmail() {
var emailValue =
document.forms["loginForm"]["email"].value;

    if (emailValue === "") {
      alert("Please enter your email.");
    } else {
      alert("Email: " + emailValue);
    }
  }
</script> </body> </html>
```


3. reset() Method

- Resets all form fields to their default values.
- Default values are the ones written in the HTML when the page

```
<html>
<body>
  <h2>Reset Form</h2>
  <form id="myForm">
    Name: <input type="text" value="Ben" ><br><br>
    Email: <input type="text" ><br><br>
    <button type="button" onclick="resetForm()">Reset</button>
  </form>
  <script>
    function resetForm() {
      document.getElementById("myForm").reset();
    }
  </script> </body> </html>
```

3. focus()

- Places the cursor inside an input field.

```
<html>
<body>
  Username: <input type="text" id="un">
  <script>
    document.getElementById("un").focus();
  </script>
</body>
</html>
```

Username:

```
<html>
<body>
  Username: <input type="text" id="un">
  <script>
    document.getElementById("un").value;
  </script>
</body>
</html>
```

Username:

Event Handler Attributes

- These are HTML attributes that respond to user actions.

1. **onchange**

- Event occurs when the value of an HTML element is changed.
- This event is commonly used with input fields, select dropdowns, and text areas.
- For text inputs → Runs after finish typing AND click outside.
- For dropdowns / checkboxes → Runs immediately when changed.

Attribute	Triggered When...
onclick	Element is clicked
ondblclick	Double-clicked
onmouseover	Mouse moves over element
onmouseout	Mouse leaves element
onkeydown	Key is pressed
onkeyup	Key is released
onchange	Input value changes
onsubmit	Form is submitted
onload	Page finishes loading

```
<body>
Enter your name:
<input type="text" id="name" onchange="showName()">
<p id="result"></p>
<script>
function showName() {
  let name = document.getElementById("name").value;
  document.getElementById("result").innerHTML = "Hello " + name;
}
</script>
```

Enter your name:

Enter your name:

Enter your name:
Hello Web

```
Enter your name:
<input type="text" id="nameBox" onchange="makeUppercase()">
<script>
function makeUppercase() {
  let textbox = document.getElementById("nameBox");
  textbox.value = textbox.value.toUpperCase();
}
</script>
```

2. onmouseover

- onmouseover is an event that runs *when your mouse pointer moves over an element.*

```
<html> <body>
<p id="text" onmouseover="changeColor()" <b>Web Programming</b> </p>
<script>
function changeColor() {
  document.getElementById("text").style.color = "red";
}
</script> </body> </html>
```

Web Programming

Web Programming

```
<body>
<p id="text" onmouseover="changeText()">Move your mouse over this text </p>
<script>
function changeText() {
  document.getElementById("text").innerHTML = "BCSE203E";
}
</script> </body>
```

← → ↻ ⓘ File E:/VIT

Move your mouse over this text

← → ↻ ⓘ File

BCSE203E

Textbox with Validation

```
<html>
<body>

<form id="myForm">
  Name: <input type="text" id="name" placeholder="Enter your name">
  <br><br>
  Email: <input type="text" id="email" placeholder="Enter your email">
  <br><br>
  Password: <input type="password" id="password" placeholder="Enter password">
  <br><br>

  <button type="submit">Submit</button>
</form>
```

Name :

Email:

Password:

```
<script>
document.getElementById("myForm").addEventListener("submit", function(event) {
    event.preventDefault();
    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;
    let password = document.getElementById("password").value;
```

```
    if (name === "") {
        alert("Name is required");
        return;
    }

    let emailPattern = /^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
    if (!email.match(emailPattern)) {
        alert("Enter a valid email");
        return;
    }

    if (password.length < 6) {
        alert("Password must be at least 6 characters");
        return;
    }

    alert("Form submitted successfully!");
});
</script> </body> </html>
```

/^ → Start of the string

[^] → Anything except a space

@ → Must contain the @ symbol.

\. → dot before .com

